Part 2. Download & R Workflow

API access with tidycensus and RSocrata, step-by-step R scripts for downloading, cleaning, and combining the toolkit datasets, followed by descriptive statistics, correlation, multiple regression, and logistic regression.

Prerequisite: Part 1 complete | **Level:** Beginner–Intermediate | **Time:** ~3–4 hours

Learning Objectives

- Set up a reproducible R project with the correct folder structure and all required packages.
- Download data using the Census API via `tidycensus`, `RSocrata`, and direct download.
- Merge all datasets into one clean master county-level CSV using `dplyr` joins.
- Run descriptive statistics, correlation matrix, and regression analyses.
- Export results, a data dictionary, and a README file for Tableau and open sharing on GitHub.

What you need before starting: install R and RStudio (free download) on your computer

- R language (base): https://cran.r-project.org/, download the version for your operating system (e.g., macOS, Windows).
- RStudio Desktop: https://posit.co/download/rstudio-desktop/, a free coding environment where you write and run all R scripts for this Part 2.
- Minimum recommended: R 4.3.0 or higher. Check your version with: `R.version$major`

1. R Environment Setup

Before running any scripts, complete the one-time setup steps below. This step needs to be completed only once per computer.

Project Folder Structure

Note: For the desktop version, begin by creating a folder on your desktop (or your desired Drive, such as C:\, D:\, or E:\ on your hard drive or USB drive) called **sdoh_toolkit**.

In RStudio, look up the RStudio Menu Bar at the top of the R Console. Then, select **File** > **New Project** > **Existing Directory.** Then, point your mouse to the desired drive containing the "**sdoh_toolkit**" folder, click the "**Browse…**" tab, and select the folder by pointing to it.

Then, click the "**Create Project**" tab.

Run the following R codes to build the Project Folder Structure without any files among all the folders:

```
# Create the entire folder structure from R (run once):
dirs <- c(
  "data/raw", "data/clean", "data/master",
  "code", "output/tableau", "output/figures", "output/tables",
  "dictionary"
)
sapply(dirs, dir.create, recursive = TRUE, showWarnings = FALSE)
cat("All folders created.\n")
```

Note. After you have completed all practices, the project folders may contain multiple files with the following Project Folder Structure with several layers of different types of files, such as data files, R codes/R scripts, R outputs, and data dictionary files as follows:

```
sdoh_toolkit/
  data/
    raw/          <- original downloaded files — NEVER modify these
    clean/        <- cleaned individual dataset files
    master/       <- final merged dataset (Script #6 output)
  code/           <- Paste each script into an R script and save it as an .R file
    01_acs.R
    02_places.R
    03_svi_eji.R
    04_hrsa.R
    05_usda_nces_chr.R
    06_merge_all.R
    07_analysis.R
  output/
    tableau/      <- final CSV for Tableau dashboard
    figures/      <- correlation plots, regression output
    tables/       <- descriptive stats, model results
  dictionary/
    data_dictionary.csv
```

Install Required R Packages

Copy and run this block once in your RStudio console. It installs every package used across all scripts.

```
# —— Step 1: Install all required packages (run once) ——————————————————————————————
install.packages(c(
  "tidycensus",   # ACS and Census API access
  "tidyverse",    # data wrangling (dplyr, ggplot2, readr, tidyr)
  "haven",        # read SAS/Stata files (.xpt, .dta)
  "readxl",       # read Excel files (.xlsx, .xls)
  "janitor",      # clean column names (clean_names())
  "skimr",        # enhanced descriptive statistics
  "corrplot",     # correlation matrix visualization
  "car",          # regression diagnostics (vif, Anova)
  "broom",        # tidy regression output (tidy(), glance())
  "lmtest",       # Breusch-Pagan test for homoscedasticity
  "ResourceSelection", # Hosmer-Lemeshow test for logistic regression
  "devtools", # required to install RSocrata from GitHub
  "sandwich", # robust standard errors
))

# —— Step 2: Install RSocrata from GitHub (run once) ————————————————————————
# RSocrata is maintained by the City of Chicago (github.com/Chicago/RSocrata)
# It is not available on CRAN for all R versions, so install from GitHub
# During installation, you may see two prompts:
# Question 1: "Would you like to install remotes?" -- select 1 (Yes)
# Question 2: "Which would you like to update?" -- select 1 (All)
#           If option 1 causes errors, re-run and select 3 (None)
devtools::install_github("Chicago/RSocrata")

# —— Step 3: Verify all packages loaded successfully ————————————————————————
# Verify all packages loaded successfully
library(tidycensus); library(tidyverse); library(RSocrata)
library(haven); library(readxl); library(janitor)
library(skimr); library(corrplot); library(car); library(broom)
library(lmtest); library(ResourceSelection)
```

Get Your Free Census API Key

The US Census API key is required for `tidycensus`, and the process takes about 2 minutes.

```
# Step 1: Request your free key at https://api.census.gov/data/key_signup.html
#         by typing your institution's name and your email address
# Step 2: Check your email. Your key arrives within a few minutes.
#         Remember to activate your API Key!
# Step 3: Install the API key in R (saves to your .Renviron; done once per machine)
library(tidycensus)
census_api_key("PUT_YOUR_KEY_HERE", install = TRUE, overwrite = TRUE)

# Step 4: Restart R first — Session menu > Restart R
# OR run this line instead of restarting:
readRenviron("~/.Renviron")

# Then verify your key is stored:
sys.getenv("CENSUS_API_KEY") # should print your key.
```

2. Data Download Scripts (Scripts #1–#5)

Scripts #1 through #5 download or prepare, clean, and save the source datasets used in the toolkit. All scripts follow the same structure: (1) load packages, (2) download via an API or direct URL, (3) clean and standardize FIPS codes, and (4) save the cleaned CSV to data/clean/. Run the scripts in order.

Script #1: ACS 5-Year for Domains 1 & 2

The American Community Survey (ACS) is the main dataset. This script uses the Census API with the `tidycensus` package to download variables for poverty, income, education, housing, insurance, and SNAP. Illinois at the county level is used as the default example throughout. Change the `state` and `geography` arguments to use any state or geographic unit.

Before running this script, you need to know the following things:

- **Where to find variable codes:** ACS has thousands of variables. Each has a code like `B17001_002`. To browse all available variables, run this in RStudio. Search by keyword (e.g., "poverty", "income", "education") to find the code for any variable you need. The Census Bureau also provides a full variable list at: api.census.gov/data/2024/acs/acs5/variables.html

  ```
  vars <- load_variables(2024, "acs5")
  View(vars)   # opens a searchable table in RStudio
  ```

- **Where to get training:** The `tidycensus` package has free official documentation and tutorials at walker-data.com/tidycensus, written by the package author Kyle Walker. The U.S. Census Bureau also provides free video tutorials on using the Census API at census.gov/data/academy.

```
# ── Script #1: ACS 5-Year Download ──────────────────────────────────────────
# Source:    U.S. Census Bureau, American Community Survey 5-year estimates
# API:       api.census.gov via tidycensus package
# Geography: County level, Illinois (state = 'IL')
# Year:      2024 (most recent 5-year ACS, 2020-2024)
# Citation:  U.S. Census Bureau. (2026). ACS 5-year estimates 2020-2024.
#            https://data.census.gov/
# ────────────────────────────────────────────────────────────────────────────
```

```r
library(tidycensus)
library(tidyverse)
library(janitor)

# ── Define variables to pull ──────────────────────────────────────────────────
# Variable codes: browse with load_variables(2024, 'acs5') in RStudio
acs_vars <- c(
  # Domain 1 — Economic Stability
  total_pop          = "B17001_001",  # total population for poverty denom
  poverty_count      = "B17001_002",  # persons below poverty line
  median_hh_income   = "B19013_001",  # median household income ($)
  unemployed         = "B23025_005",  # unemployed persons in labor force
  labor_force        = "B23025_002",  # total labor force
  snap_hh            = "B22010_002",  # households receiving SNAP
  total_hh           = "B22010_001",  # total households (SNAP denom)
  rent_burden_30plus = "B25070_007",  # gross rent 30-34.9% of income
  rent_burden_35plus = "B25070_008",  # gross rent 35-39.9% of income
  rent_burden_40plus = "B25070_009",  # gross rent 40-49.9% of income
  rent_burden_50plus = "B25070_010",  # gross rent 50%+ of income
  rent_denom         = "B25070_001",  # renter-occupied units (denom)

  # Domain 2 — Education Access & Quality
  edu_total_25plus   = "B15003_001",  # total pop 25+ (education denom)
  no_hs_diploma      = "B15003_002",  # less than 9th grade
  hs_graduate        = "B15003_017",  # HS graduate or equivalent
  bachelors          = "B15003_022",  # bachelor's degree
  graduate           = "B15003_023",  # master's degree
  professional       = "B15003_024",  # professional school degree
  doctorate          = "B15003_025",  # doctorate degree

  # Domain 3 — cross-domain variable (health insurance)
  uninsured_pct = "DP03_0099P"  # % of residents without health insurance
                                # excludes prisoners and nursing home residents
)

# ── Download from Census API ───────────────────────────────────────────────────
acs_raw <- get_acs(
  geography = "county",
  variables = acs_vars,
  state     = "IL",          # Change to 'all' for national data
  year      = 2024,
  survey    = "acs5",
  output    = "wide"          # one row per county, all vars as columns
)

# ── Clean and compute derived rates ──────────────────────────────────────────
acs_clean <- acs_raw %>%
  clean_names() %>%
  rename(county_name = name) %>%
  mutate(
    # Standardize FIPS: always 5-digit character string with leading zeros
    fips = str_pad(geoid, width = 5, side = "left", pad = "0"),

    # Compute rates from raw counts
    poverty_rate     = round(poverty_count_e / total_pop_e * 100, 2),
    unemployment_rate= round(unemployed_e   / labor_force_e * 100, 2),
    snap_rate        = round(snap_hh_e       / total_hh_e   * 100, 2),
```

```
    # Rent cost burden: share of renters paying 30%+ of income on rent
    rent_burden_rate = round(
      (rent_burden_30plus_e + rent_burden_35plus_e +
       rent_burden_40plus_e + rent_burden_50plus_e) / rent_denom_e * 100, 2),

    # Education rates (% of adults 25+)
    pct_no_hs = round(no_hs_diploma_e / edu_total_25plus_e * 100, 2),
    pct_hs_grad = round(hs_graduate_e   / edu_total_25plus_e * 100, 2),
    pct_bachelor_plus = round(
      (bachelors_e + graduate_e + professional_e + doctorate_e) /
       edu_total_25plus_e * 100, 2),

    # Uninsured rate
    uninsured_rate = uninsured_pct_e   # already a percentage from DP03_0099P
  ) %>%
  select(fips, county_name, poverty_rate, median_hh_income_e,
         unemployment_rate, snap_rate, rent_burden_rate,
         pct_no_hs, pct_hs_grad, pct_bachelor_plus, uninsured_rate) %>%
  rename(median_hh_income = median_hh_income_e)

# ── Inspect the output ──────────────────────────────────────────
glimpse(acs_clean)
summary(acs_clean)

# ── Save to data/clean/ ─────────────────────────────────────────
write_csv(acs_clean, "data/clean/01_acs_clean.csv")
cat("Script #1 complete. Rows:", nrow(acs_clean), "\n")
```

You can use different geographies to conduct your analyses based on your research questions.

- **To change the geographic unit:** Replace geography = "county" with `geography = "tract"` for census tract data, or `geography = "zcta"` for ZIP code data.
- **To switch to national data:** Change `state = 'IL'` to `state = NULL` or remove the state argument entirely to pull all U.S. counties. The number of rows depends on your chosen geography: approximately 3,100 for county, 85,000 for census tract, and 33,000 for ZCTA.

Script #2: CDC PLACES 2025 for Domain 3

CDC PLACES provides model-based estimates for 40 health outcomes and behaviors. This script downloads the data directly into R using base R `read.csv()` from the CDC public API endpoint. No additional package or app token is required. Again, Illinois at the county level is used as the default example throughout.

```
# ─── Script #2: CDC PLACES 2025 County Data ──────────────────────
# Source:    Centers for Disease Control and Prevention
# Endpoint:  https://data.cdc.gov/resource/swc5-untb.csv
# Geography: County level — Illinois default. See end of script for tract/ZCTA
# Citation:  CDC. (2025). PLACES: Local data for better health, county data
#            2025 release. https://data.cdc.gov/resource/swc5-untb
# ─────────────────────────────────────────────────────────────────

library(tidyverse)
library(janitor)
```

```
# ── Download PLACES data — Illinois counties only ──────────────────────────────
places_raw <- read.csv(
  "https://data.cdc.gov/resource/swc5-untb.csv?StateAbbr=IL&$limit=10000",
  stringsAsFactors = FALSE
)

# ── Check the download ─────────────────────────────────────────────────────────
cat("Total rows downloaded:", nrow(places_raw), "\n")
names(places_raw)
unique(places_raw$measureid) %>% sort()

# ── Check county coverage per measure ──────────────────────────────────────────
─────────────────────────
places_raw %>%
  filter(datavaluetypeid == "CrdPrv") %>%
  group_by(measureid) %>%
  summarise(n_counties = n()) %>%
  arrange(n_counties) %>%
  View()

# ── Optional: Browse available measures ────────────────────────────────────────
# Run these lines to explore what measures are available before choosing
# Browse all measures with descriptions via searchable table in RStudio
places_raw %>%
  select(measureid, measure, category) %>%
  distinct() %>%
  arrange(category, measureid) %>%
  View()

# Search by keyword — change "diabetes" to any topic you are interested in
places_raw %>%
  select(measureid, measure, category) %>%
  distinct() %>%
  filter(str_detect(tolower(measure), "diabetes"))

# ── Select key measures and pivot to wide format ───────────────────────────────
measures_keep <- c(
  "DIABETES",
  "OBESITY",
  "BPHIGH",
  "DEPRESSION",
  "COPD",
  "CASTHMA",
  "CSMOKING",
  "LPA",
  "COLON_SCREEN",
  "MAMMOUSE",
  "ACCESS2"
)

places_wide <- places_raw %>%
  filter(measureid %in% measures_keep) %>%
  filter(datavaluetypeid == "CrdPrv") %>%
  select(locationid, locationname, measureid, data_value) %>%
  mutate(
    fips       = str_pad(locationid, width = 5, side = "left", pad = "0"),
    data_value = as.numeric(data_value)
```

```
  ) %>%
  pivot_wider(
    id_cols     = c(fips, locationname),
    names_from  = measureid,
    values_from = data_value
  ) %>%
  clean_names() %>%
  rename(
    diabetes_pct          = diabetes,
    obesity_pct           = obesity,
    hypertension_pct      = bphigh,
    depression_pct        = depression,
    copd_pct              = copd,
    asthma_pct            = casthma,
    smoking_pct           = csmoking,
    inactivity_pct        = lpa,
    colorectal_screen_pct = colon_screen,
    mammography_pct       = mammouse,
    uninsured_places      = access2
  )

cat("Counties in PLACES data:", nrow(places_wide), "\n")
glimpse(places_wide)
write_csv(places_wide, "data/clean/02_places_clean.csv")
cat("Script #2 complete. Rows:", nrow(places_wide), "\n")
```

Here's the information you need to download PLACES data at the census tract or ZCTA level, depending on your research question.

Census Tract Level
The tract file is already in wide format with one row per tract and all measures as separate columns named like `diabetes_crudeprev`. You do not need `pivot_wider()`. Use `tractfips` as the geographic identifier.

```
# ─── Census tract and ZCTA options ───────────────────────────────────────────────
# Census tract — wide format, one row per tract, no pivot_wider() needed
# Use tractfips as the geographic identifier
places_tract <- read.csv(
  "https://data.cdc.gov/resource/yjkw-uj5s.csv?StateAbbr=IL&$limit=300000",
  stringsAsFactors = FALSE
)
names(places_tract)
```

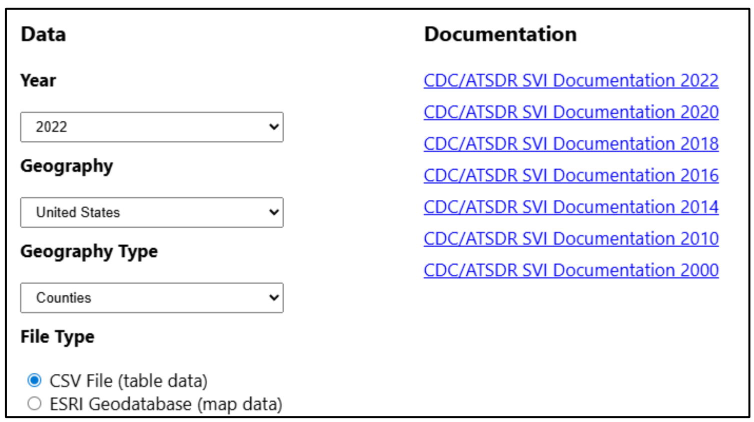
ZCTA (ZIP Code Tabulation Area)
The ZCTA file is also in wide format. Use `zcta5` as the geographic identifier. ZCTAs do not align perfectly with county or tract boundaries; merging ZCTA data with other datasets requires a ZCTA-to-county crosswalk.

```
install.packages("CDCPLACES")
library(CDCPLACES)

il_zcta <- get_places(
  geography = "zcta",
  state     = "IL",
  measure   = c("DIABETES", "OBESITY", "BPHIGH",
                "DEPRESSION", "COPD", "CASTHMA",
```

```
                 "CSMOKING", "LPA", "COLON_SCREEN",
                 "MAMMOUSE", "ACCESS2")
)

cat("Illinois ZCTA rows:", nrow(il_zcta), "\n")

# View structure and first few rows
glimpse(il_zcta)

# View column names
names(il_zcta)

# View first 10 rows in console
head(il_zcta, 10)

# Open searchable table in RStudio
View(il_zcta)

# Check available measures
unique(il_zcta$measure) %>% sort()
```

See full documentation at: `brendensm.github.io/CDCPLACES`

Script #3: CDC/ATSDR SVI 2022 & EJI 2024 for Domains 4 & 5
This script reads the SVI and EJI directly from downloaded CSV files from the ATSDR website. Download both files into data/raw/ before running. SVI offers county-level social vulnerability across 4 themes. The EJI provides tract-level environmental, social, and health vulnerability. The script aggregates tract-level EJI percentile values to the county level using unweighted means.



SVI 2022 County CSV: Save as: data/raw/SVI2022_US_county.csv

EJI 2024 CSV and file type should be .csv file. After the download is complete, extract the ZIP file and locate **EJI_2024_United_States.csv.** Save and rename this file as: data/raw/EJI_2024_US.csv

Note: If using the tract-level SVI file, the state column is named `stabbr` not `st_abbr`. Change the filter accordingly: `filter(stabbr == "IL")`.

```
# ── Script #3: SVI 2022 & EJI 2024 ─────────────────────────────────────────
# SVI Source:  ATSDR. (2023). SVI 2022.
#              https://www.atsdr.cdc.gov/place-health/php/svi/index.html
# EJI Source:  CDC/ATSDR. (2024). Environmental Justice Index (EJI).
#              https://www.atsdr.cdc.gov/place-health/php/eji/index.html
# ──────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(janitor)

# ══ PART A: Social Vulnerability Index (SVI) ════════════════════════════════
```

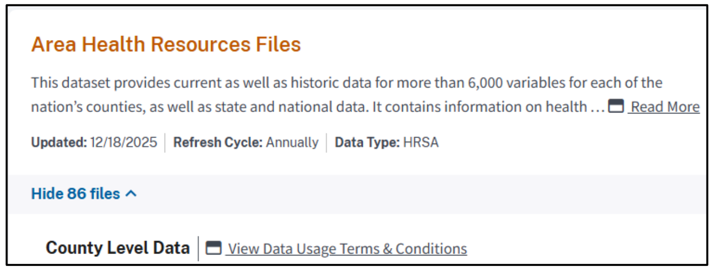
```r
svi_raw <- read_csv("data/raw/SVI2022_US_county.csv")

svi_clean <- svi_raw %>%
  clean_names() %>%
  filter(st_abbr == "IL") %>%     # Change 'IL' to filter a different state
  mutate(
    # Pad FIPS to 5 digits — critical for merging
    fips = str_pad(as.character(fips), width = 5, side = "left", pad = "0"),

    # SVI flag: -999 means no data — replace with NA
    across(starts_with("rpl_"), ~ ifelse(. == -999, NA, .))
  ) %>%
  select(
    fips,
    county_name  = county,
    svi_overall  = rpl_themes,   # Overall SVI percentile (0-1; higher=more
vulnerable)
    svi_socioeco = rpl_theme1,   # Theme 1: Socioeconomic status
    svi_hh_char  = rpl_theme2,   # Theme 2: Household characteristics
    svi_minority = rpl_theme3,   # Theme 3: Racial/ethnic minority & language
    svi_housing  = rpl_theme4,   # Theme 4: Housing type & transportation
    # Key component variables
    pct_poverty  = ep_pov150,    # % persons below 150% poverty line
    pct_unemp_svi= ep_unemp,     # % civilian unemployed
    pct_uninsured_svi = ep_uninsur  # % uninsured
  )

cat("SVI rows (IL counties):", nrow(svi_clean), "\n")
write_csv(svi_clean, "data/clean/03a_svi_clean.csv")

# ══ PART B: Environmental Justice Index (EJI) ════════════════════════════════
# EJI is tract-level — aggregate to county by averaging

eji_raw <- read_csv("data/raw/EJI_2024_US.csv")

eji_county <- eji_raw %>%
  clean_names() %>%
  mutate(
    fips       = str_sub(geoid, 1, 5),
    state_abbr = stateabbr
  ) %>%
  filter(state_abbr == "IL") %>%
  mutate(across(c(rpl_eji, rpl_ebm, rpl_svm, rpl_hvm),
                ~ ifelse(. == -999, NA, .))) %>%
  group_by(fips) %>%
  summarise(
    eji_overall    = round(mean(rpl_eji, na.rm=TRUE), 4),  # Overall EJI
    eji_env_burden = round(mean(rpl_ebm, na.rm=TRUE), 4),  # Environmental Burden
    eji_social_vuln= round(mean(rpl_svm, na.rm=TRUE), 4),  # Social Vulnerability
    eji_health_vuln= round(mean(rpl_hvm, na.rm=TRUE), 4),  # Health Vulnerability
    n_tracts       = n()
  ) %>%
  ungroup()

cat("EJI county rows (IL):", nrow(eji_county), "\n")
write_csv(eji_county, "data/clean/03b_eji_county_clean.csv")
```

```
cat("Script #3 complete.\n")
```

Script #4: HRSA HPSA Shortage Areas for Domain 3
This script reads Health Professional Shortage Area (HPSA) designation data from the Area Health Resources Files (AHRF), published by HRSA. The AHRF is a county-level file that provides shortage designation status for primary care, dental, and mental health across all U.S. counties. Each county receives a designation code for each shortage type: 0 means no shortage designation, 1 means part of the county is designated, and 2 means the whole county is designated.

Before running this script, download the AHRF file:

1. Go to data.hrsa.gov/data/download
2. Click **Area Health Resources Files**
3. Under County, download **AHRF 2024-2025 County CSV**
4. Unzip the file and save AHRF2025.csv to data/raw/AHRF2025.csv



Note: AHRF is county-level only and cannot be broken down to census tract or ZIP code. If changed to tract-level geography in other scripts, omit Script #4 and remove the HPSA join from Script #6.

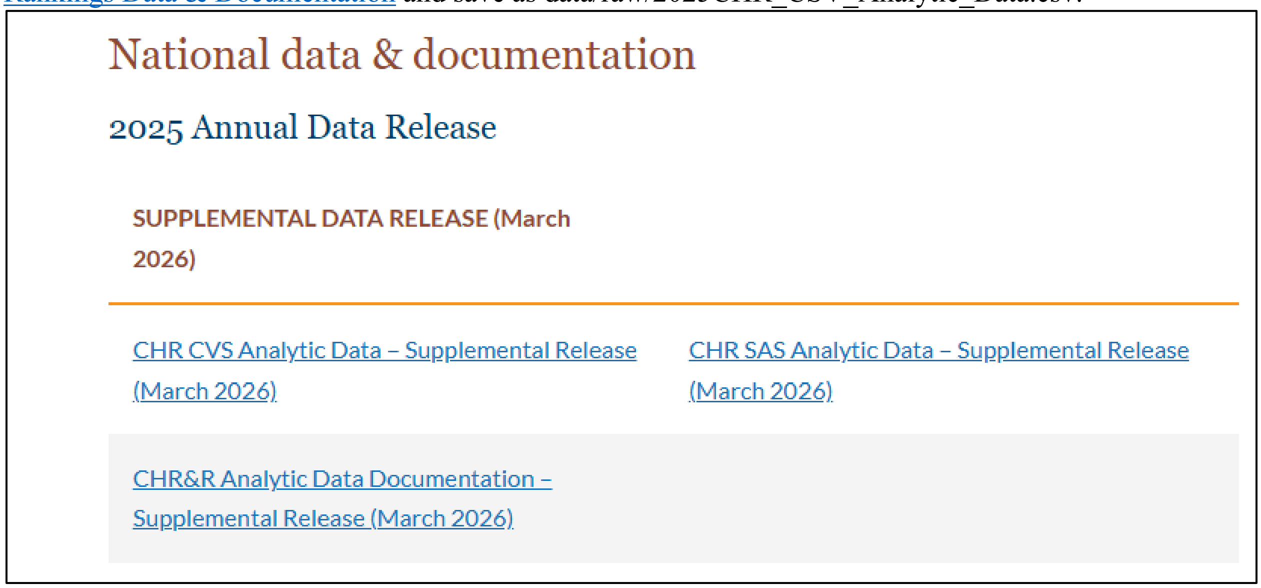
```
# ── Script #4: HRSA HPSA Shortage Areas from AHRF ─────────────────────────────
# Source:  Health Resources & Services Administration
#          Area Health Resources Files (AHRF) 2024-2025
# URL:     https://data.hrsa.gov/data/download
# Steps:   1. Click "Area Health Resources Files"
#          2. Under County, download: AHRF 2024-2025 County CSV
#          3. Unzip and save AHRF2025.csv as: data/raw/AHRF2025.csv
# Citation: HRSA, Bureau of Health Workforce. (2025). Area Health Resources
#           Files 2024-2025. https://data.hrsa.gov/data/download
# Note:    AHRF is county-level only. Cannot be disaggregated to tract or ZIP.
#          HPSA variables use designation codes:
#          0 = No shortage designation
#          1 = Part of county is designated (partial shortage)
#          2 = Whole county is designated (full shortage)
# ──────────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(janitor)

# ── Load AHRF county file ─────────────────────────────────────────────────────
ahrf_raw <- read_csv("data/raw/AHRF2025.csv")

# ── Inspect column names ──────────────────────────────────────────────────────
names(ahrf_raw) %>% sort()

# ── Filter to Illinois and select HPSA variables ──────────────────────────────
hpsa_clean <- ahrf_raw %>%
  clean_names() %>%
  filter(st_name_abbrev == "IL") %>%   # Change "IL" for other states
  mutate(
    fips = str_pad(as.character(fips_st_cnty), 5, "left", "0")
```

```
  ) %>%
  select(
    fips,
    county_name         = cnty_name,
    # 2025 HPSA designation codes (0=none, 1=partial, 2=full)
    hpsa_prim_care      = hpsa_prim_care_25,   # Primary care shortage
    hpsa_dental         = hpsa_dent_25,         # Dental shortage
    hpsa_mental_health  = hpsa_mentl_hlth_25    # Mental health shortage
  ) %>%
  mutate(
    # Convert to numeric
    across(c(hpsa_prim_care, hpsa_dental, hpsa_mental_health), as.numeric),
    # Create binary shortage indicator (1 = any shortage, 0 = none)
    hpsa_any_shortage   = as.integer(
      hpsa_prim_care > 0 | hpsa_dental > 0 | hpsa_mental_health > 0),
    # Create primary care binary (used in logistic regression)
    hpsa_prim_care_designated = as.integer(hpsa_prim_care > 0)
  )

# ── Check distribution ────────────────────────────────────────────────
cat("HPSA rows (IL counties):", nrow(hpsa_clean), "\n")

# Primary care shortage distribution
cat("\nPrimary care HPSA designation (0=none, 1=partial, 2=full):\n")
table(hpsa_clean$hpsa_prim_care)

cat("\nDental HPSA designation:\n")
table(hpsa_clean$hpsa_dental)

cat("\nMental health HPSA designation:\n")
table(hpsa_clean$hpsa_mental_health)

cat("\nCounties with ANY shortage designation:",
    sum(hpsa_clean$hpsa_any_shortage), "\n")

# ── Save ──────────────────────────────────────────────────────────────
write_csv(hpsa_clean, "data/clean/04_hpsa_clean.csv")
cat("Script #4 complete.\n")
```

Script #5: USDA Food Access, NCES CCD & County Health Rankings for Domains 1, 2 & 5
This script reads three datasets covering food access, education, and community health.

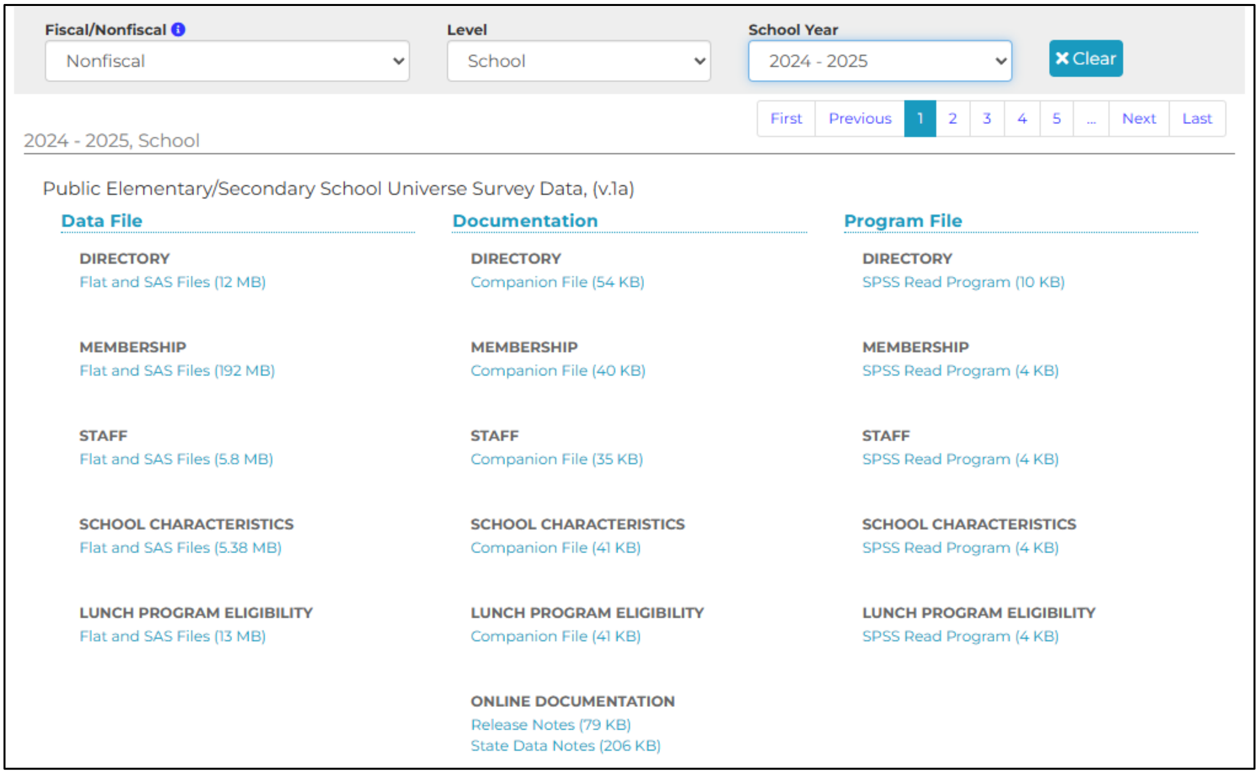
The USDA Food Access Research Atlas provides food desert classifications at the census tract level, aggregated to the county level. The most current USDA Food Access data year is 2019. Download the Excel file from [Food Access Research Atlas]() and save as data/raw/FoodAccessResearchAtlasData.xlsx

County Health Rankings & Roadmaps (CHR&R) provides pre-calculated county-level health outcomes and civic engagement variables. Download the 2025 national data CSV from County Health Rankings Data & Documentation and save as data/raw/2025CHR_CSV_Analytic_Data.csv.

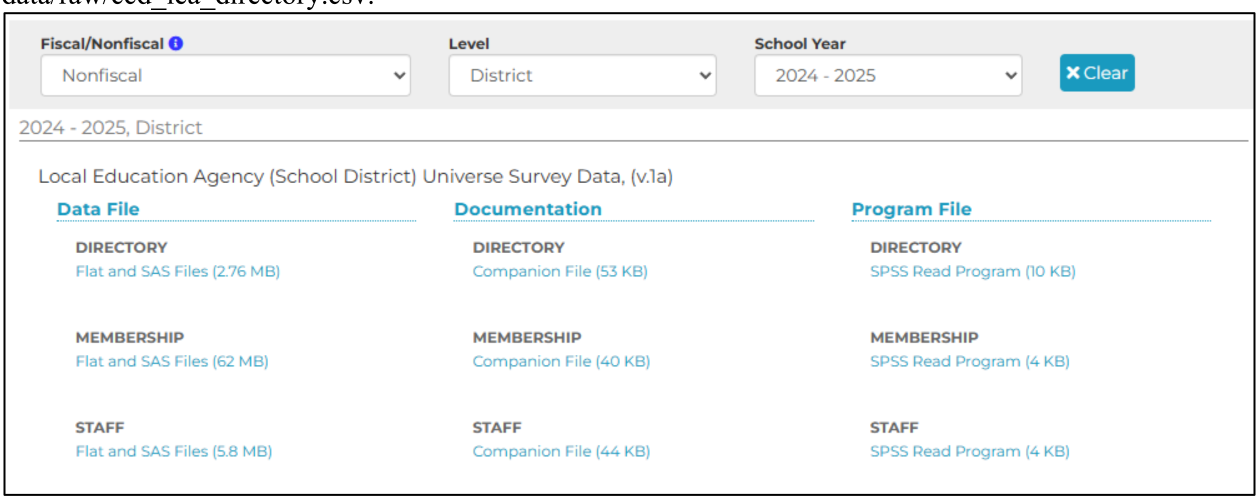


The NCES Common Core of Data (CCD) provides school enrollment data by district, aggregated to the county level. Two files are required. Go to nces.ed.gov/ccd/files.asp and download:

(1) Nonfiscal > School > 2024-2025 > LUNCH PROGRAM ELIGIBILITY (13 MB): save CSV as data/raw/ccd_lunch.csv



(2) Nonfiscal > District > 2024-2025 > DIRECTORY (2.76 MB): save CSV as data/raw/ccd_lea_directory.csv.



Note. Illinois does not report free and reduced-price lunch counts, but 46 other states do. This script automatically detects whether your state reports this data.

```
# ── Script #5: USDA Food Access + NCES CCD + County Health Rankings ──────────
# Download before running:
# USDA: ers.usda.gov/data-products/food-access-research-atlas/download-the-data
#       Save as: data/raw/FoodAccessResearchAtlasData.xlsx
#       Note: Most current data year is 2019.
# CHR&R: countyhealthrankings.org/health-data/methodology-and-sources/data-
documentation
```

```r
#        Save as: data/raw/2025CHR_CSV_Analytic_Data.csv
# NCES CCD: nces.ed.gov/ccd/files.asp(District file, most recent year)
#   Download 1: Nonfiscal > School > 2024-2025 > LUNCH PROGRAM ELIGIBILITY
#               Unzip and save CSV as: data/raw/ccd_lunch.csv
#   Download 2: Nonfiscal > District > 2024-2025 > DIRECTORY
#               Unzip and save CSV as: data/raw/ccd_lea_directory.csv
#   Note: Free/reduced lunch counts not available for all states.
#         Illinois does not report these. 46 other states do.
#   Note: ZIP-to-county crosswalk may assign some districts to multiple
#         counties. Enrollment counts are used as descriptive context only.
# ────────────────────────────────────────────────────────────────────────────

library(tidyverse)
library(readxl)
library(janitor)

# ═══ PART A: USDA Food Access Research Atlas ═══════════════════════════════
usda_raw <- read_excel(
  "data/raw/FoodAccessResearchAtlasData.xlsx",
  sheet = "Food Access Research Atlas"
)

# ─── Check column names before cleaning ────────────────────────────────────
usda_raw %>% clean_names() %>% names() %>%
  .[grep("lila|lapop|lahunv|tract|state", .)] %>%
  head(20)


# ── Then run the cleaning code ──────────────────────────────────────────────
usda_county <- usda_raw %>%
  clean_names() %>%
  filter(state == "Illinois") %>%
  mutate(
    fips = str_pad(as.character(census_tract), 11, "left", "0") %>%
           str_sub(1, 5),
    lapophalfshare = as.numeric(lapophalfshare),
    lahunv1share   = as.numeric(lahunv1share)
  ) %>%
  group_by(fips) %>%
  summarise(
    pct_lila_tracts       = round(mean(lila_tracts_1and10, na.rm=TRUE) * 100, 2),
    mean_dist_supermarket = round(mean(lapophalfshare,      na.rm=TRUE), 4),
    pct_no_vehicle_far    = round(mean(lahunv1share,        na.rm=TRUE) * 100, 2)
  ) %>%
ungroup()

cat("USDA county rows:", nrow(usda_county), "\n")
write_csv(usda_county, "data/clean/05a_usda_food_access_clean.csv")

# ═══ PART B: County Health Rankings 2025 ═══════════════════════════════════
chr_raw <- read_csv(
  "data/raw/2025CHR_CSV_Analytic_Data.csv",
  skip = 1
)
chr_clean <- chr_raw %>%
  clean_names() %>%
  filter(state == "IL",
```

```
         fipscode != "17000") %>%       # remove state summary row
  mutate(
    fips = str_pad(as.character(fipscode), 5, "left", "0")
  ) %>%
  select(
    fips,
    premature_death     = v001_rawvalue,
    poor_health_days    = v036_rawvalue,
    poor_mental_days    = v042_rawvalue,
    social_associations = v140_rawvalue,
    voter_turnout       = v153_rawvalue,
    adult_smoking       = v009_rawvalue,
    adult_obesity       = v011_rawvalue,
    food_insecurity     = v139_rawvalue
  ) %>%
  mutate(
    across(everything(), as.numeric),
    # Convert proportions to percentages
    voter_turnout   = voter_turnout   * 100,
    adult_smoking   = adult_smoking   * 100,
    adult_obesity   = adult_obesity   * 100,
    food_insecurity = food_insecurity * 100
  )

# Note: Column codes change with each annual CHR&R release.
# Run names(chr_raw) and check the CHR codebook to verify codes.
cat("CHR&R county rows:", nrow(chr_clean), "\n")
write_csv(chr_clean, "data/clean/05b_chr_clean.csv")

# ═══ PART C: NCES CCD Lunch Program Eligibility ══════════════════════════════════

# ─── Set your state FIPS here ──────────────────────────────────────────────────
STATE_FIPS <- "17"   # Illinois = 17. Change for other states.

# ─── Download Census TIGER ZIP-to-county crosswalk ─────────────────────────────
tiger_url <- "https://www2.census.gov/geo/docs/maps-
data/data/rel2020/zcta520/tab20_zcta520_county20_natl.txt"

zip_county_xwalk <- read.delim(tiger_url, sep = "|") %>%
  clean_names() %>%
  select(geoid_zcta5_20, geoid_county_20) %>%
  rename(zip = geoid_zcta5_20, fips = geoid_county_20) %>%
  mutate(
    zip  = as.character(zip),
    fips = as.character(fips)
  ) %>%
  filter(str_sub(fips, 1, 2) == STATE_FIPS) %>%
  distinct()

cat("ZIP-county pairs downloaded:", nrow(zip_county_xwalk), "\n")

# ─── Load district directory — use ZIP to get county FIPS ──────────────────────
dir_raw <- read.csv("data/raw/ccd_lea_directory.csv",
                    stringsAsFactors = FALSE) %>%
  clean_names() %>%
  filter(as.character(fipst) == STATE_FIPS) %>%
  mutate(
```

```r
    leaid = as.character(leaid),
    zip   = str_pad(as.character(mzip), 5, "left", "0")
  )

# Valid IL county FIPS — Illinois uses odd numbers only (17001 to 17203)
valid_il_fips <- sprintf("17%03d", seq(1, 203, by = 2))

# Join ZIP crosswalk — allow many-to-many for full county coverage
ccd_dir <- dir_raw %>%
  left_join(zip_county_xwalk, by = "zip",
            relationship = "many-to-many") %>%
  filter(!is.na(fips), fips %in% valid_il_fips) %>%
  select(leaid, fips) %>%
  distinct()

cat("Districts with valid county FIPS:", nrow(ccd_dir), "\n")
cat("Unique counties:", n_distinct(ccd_dir$fips), "\n")

# ── Load lunch file ──────────────────────────────────────────
ccd_lunch <- read.csv("data/raw/ccd_lunch.csv",
                      stringsAsFactors = FALSE) %>%
  clean_names() %>%
  filter(as.character(fipst) == STATE_FIPS)

cat("School lunch rows:", nrow(ccd_lunch), "\n")

# ── Check if your state reports free lunch counts ────────────
lunch_check <- ccd_lunch %>%
  filter(lunch_program == "Free lunch qualified",
         total_indicator == "Category Set A",
         !is.na(student_count)) %>%
  nrow()

cat("Schools with free lunch counts:", lunch_check, "\n")

if (lunch_check == 0) {
  cat("NOTE: Your state does not report free lunch counts in this file.\n")
  cat("Only total enrollment and district count will be available.\n")
  cat("Food insecurity is captured via CHR&R food_insecurity variable.\n")
} else {
  cat("Free lunch data available. Full aggregation will proceed.\n")
}

# ── Extract total enrollment ─────────────────────────────────
total_enroll <- ccd_lunch %>%
  filter(data_group == "Free and Reduced-price Lunch Table",
         lunch_program == "No Category Codes",
         total_indicator == "Education Unit Total") %>%
  select(ncessch, leaid, student_count) %>%
  rename(total_students = student_count) %>%
  mutate(
    total_students = as.numeric(total_students),
    leaid          = as.character(leaid)
  )

# ── Extract free lunch counts ────────────────────────────────
free_lunch <- ccd_lunch %>%
```

```
  filter(data_group == "Free and Reduced-price Lunch Table",
         lunch_program == "Free lunch qualified",
         total_indicator == "Category Set A") %>%
  select(ncessch, student_count) %>%
  rename(free_lunch_count = student_count) %>%
  mutate(free_lunch_count = as.numeric(free_lunch_count))

# ── Extract reduced lunch counts ─────────────────────────────────────────────
reduced_lunch <- ccd_lunch %>%
  filter(data_group == "Free and Reduced-price Lunch Table",
         lunch_program == "Reduced-price lunch qualified",
         total_indicator == "Category Set A") %>%
  select(ncessch, student_count) %>%
  rename(reduced_lunch_count = student_count) %>%
  mutate(reduced_lunch_count = as.numeric(reduced_lunch_count))

# ── Join at school level ─────────────────────────────────────────────────────
school_lunch <- total_enroll %>%
  left_join(free_lunch,    by = "ncessch") %>%
  left_join(reduced_lunch, by = "ncessch")

# ── Join district directory to get county FIPS ───────────────────────────────
school_county <- school_lunch %>%
  mutate(leaid = as.character(leaid)) %>%
  left_join(ccd_dir, by = "leaid",
            relationship = "many-to-many")

cat("Schools with county FIPS:", sum(!is.na(school_county$fips)), "\n")
cat("Unique counties in schools:", n_distinct(school_county$fips, na.rm=TRUE),
"\n")

# ── Load valid county FIPS from ACS for filtering ────────────────────────────
acs_fips <- read_csv("data/clean/01_acs_clean.csv",
                     col_types = cols(fips = col_character())) %>%
  pull(fips)

# ── Aggregate to county level ────────────────────────────────────────────────
ccd_county <- school_county %>%
  filter(!is.na(fips), fips %in% acs_fips) %>%
  group_by(fips) %>%
  summarise(
    n_districts      = n_distinct(leaid),
    total_enrollment = sum(total_students,      na.rm = TRUE),
    free_lunch       = sum(free_lunch_count,     na.rm = TRUE),
    reduced_lunch    = sum(reduced_lunch_count,  na.rm = TRUE)
  ) %>%
  ungroup() %>%
  mutate(
    pct_free_lunch    = ifelse(lunch_check > 0,
                               round(free_lunch / total_enrollment * 100, 2),
                               NA_real_ ),
    pct_reduced_lunch = ifelse(lunch_check > 0,
                               round(reduced_lunch / total_enrollment * 100, 2),
                               NA_real_ )
  ) %>%
  select(fips, n_districts, total_enrollment, pct_free_lunch, pct_reduced_lunch)
```

```
cat("NCES CCD county rows:", nrow(ccd_county), "\n")
glimpse(ccd_county)
write_csv(ccd_county, "data/clean/05c_nces_ccd_clean.csv")
cat("Script #5 complete. Rows:", nrow(ccd_county), "\n")
```

3. Merge All Datasets into Master CSV (Script 6)

The following script merges the eight cleaned source files produced by Scripts #1 through #5 into a single master analysis CSV. This master file is the input for Tableau (Part 3) and for all statistical analyses in Section 4 below.

Note: CHR&R, HRSA, and NCES data are county-level only. If you change to tract or ZCTA geography in Scripts #1–#3, remove those joins from this script and exclude their variables from your analysis. Additionally, the NCES file uses a ZIP-to-county crosswalk, which may slightly inflate enrollment counts for counties with districts on county boundaries. This is expected and does not affect other variables.

```
# ── Script #6: Merge All Datasets — Master County CSV ──────────────────────
# Input:  data/clean/01 through 05 CSV files
# Output: data/master/sdoh_il_master.csv
# ───────────────────────────────────────────────────────────────────────────

library(tidyverse)

# ── Load all clean files ─────────────────────────────────────────────────────
acs      <- read_csv("data/clean/01_acs_clean.csv")
places   <- read_csv("data/clean/02_places_clean.csv")
svi      <- read_csv("data/clean/03a_svi_clean.csv")
eji      <- read_csv("data/clean/03b_eji_county_clean.csv")
hpsa     <- read_csv("data/clean/04_hpsa_clean.csv")
usda     <- read_csv("data/clean/05a_usda_food_access_clean.csv")
chr      <- read_csv("data/clean/05b_chr_clean.csv")
nces <- read_csv("data/clean/05c_nces_ccd_clean.csv")

# ── Standardize all FIPS columns as character ──────────────────────────────
acs   <- acs    %>% mutate(fips = as.character(fips))
places <- places %>% mutate(fips = as.character(fips))
svi   <- svi    %>% mutate(fips = as.character(fips))
eji   <- eji    %>% mutate(fips = as.character(fips))
hpsa  <- hpsa   %>% mutate(fips = as.character(fips))
usda  <- usda   %>% mutate(fips = as.character(fips))
chr   <- chr    %>% mutate(fips = as.character(fips))
nces  <- nces   %>% mutate(fips = as.character(fips))

# ── Merge all on FIPS (left join — keep all ACS counties as base) ────────────
# Check row counts before and after each join to detect FIPS mismatches
cat("ACS base rows:", nrow(acs), "\n")

master <- acs %>%
  left_join(places %>% select(-locationname), by = "fips") %>%
  { cat("After PLACES:", nrow(.), "\n"); . } %>%
  left_join(svi    %>% select(-county_name),  by = "fips") %>%
  { cat("After SVI:",    nrow(.), "\n"); . } %>%
  left_join(eji,                              by = "fips") %>%
  { cat("After EJI:",    nrow(.), "\n"); . } %>%
  left_join(hpsa   %>% select(-county_name),  by = "fips") %>%
```

```
  { cat("After HPSA:",   nrow(.), "\n"); . } %>%
  left_join(usda,                             by = "fips") %>%
  { cat("After USDA:",   nrow(.), "\n"); . } %>%
  left_join(chr,                              by = "fips") %>%
  { cat("After CHR:",    nrow(.), "\n"); . } %>%
  left_join(nces,                             by = "fips") %>%
  { cat("After NCES:",   nrow(.), "\n"); . }

# ── Verify: row count should equal number of IL counties (102) ──────────────
stopifnot(nrow(master) == nrow(acs))   # stops script if rows changed

# ── Quick data quality check ─────────────────────────────────────────────
cat("\nMissing values per column:\n")
colSums(is.na(master)) %>% sort(decreasing = TRUE) %>% head(15) %>% print()

# ─── Set output file label ──────────────────────────────────────────────
# Change "il" to your state abbreviation if using a different state
STATE_LABEL <- "il"   # e.g. "ca" for California, "tx" for Texas

# ─── Save master file ───────────────────────────────────────────────────
master_filename <- paste0("data/master/sdoh_", STATE_LABEL, "_master.csv")
tableau_filename <- paste0("output/tableau/sdoh_", STATE_LABEL, "_tableau.csv")

write_csv(master, master_filename)
write_csv(master, tableau_filename)

cat("\nScript #6 complete.\n")
cat("Master file saved:", master_filename, "\n")
cat("Tableau file saved:", tableau_filename, "\n")
cat("Rows:", nrow(master), "| Columns:", ncol(master), "\n")
```

Diagnosing FIPS Merge Problems

If row counts change after a join, you have a FIPS mismatch. Common causes: (1) FIPS stored as numeric in one file (losing leading zeros), (2) FIPS is 4 digits in one file instead of 5, (3) a dataset has fewer counties than expected. Fix: always run `str_pad(as.character(fips), 5, 'left', '0')` on the FIPS column in every cleaning script before merging.

4. Statistical Analyses

This section explains the concepts to understand before running your statistical analysis, including the purpose of each method, appropriate usage scenarios, the required assumptions, and methods for verifying those assumptions.

Requirement Before Starting

Script #6 must have run successfully and produced your master CSV file in data/master/. The file name reflects your chosen state. For example, if your state is Illinois, your file is named sdoh_il_master.csv. All analyses in this section read from that file. Verify your file exists and has the correct row count by running:

```
STATE_LABEL <- "il"   # change to match your state
master <- read_csv(paste0("data/master/sdoh_", STATE_LABEL, "_master.csv"))
cat("Rows:", nrow(master), "| Columns:", ncol(master), "\n")
```

Statistical Foundations

This toolkit introduces the most commonly used statistical methods to analyze how SDOH relate to community outcomes. These methods can support research questions across different fields. For example, "Is poverty associated with higher diabetes rates?" or "Which SDOH factors best predict poor mental health outcomes?" Statistical method selection should be guided by your research question and the type of data you have.

What is Regression Analysis?
Regression analysis is a statistical method that models the relationship between an outcome variable (dependent variable) and one or more predictor variables (independent variables). In SDOH research, we generally use regression for two primary purposes: Explanation or Prediction.

Explanatory Questions: Use this approach when you want to understand how specific SDOH factors relate to a health outcome, while accounting for confounding variables. You can focus on a single primary exposure or a set of exposures. Examples include: "Is a higher poverty rate linked to increased diabetes prevalence, after adjusting for education level and insurance coverage?" and "Are counties with low civic engagement more likely to have poor mental health outcomes?" The question format is:

> *"Are [Independent Variable(s) of Interest] associated with [Dependent Variable], after controlling for [Confounders/Covariates]?"*

Predictive Questions: Use this approach when you want to identify which combination of multiple SDOH factors best forecasts or explains the variance in a population health outcome. Example includes: "Which combination of SDOH factors best explains and predicts variation in premature death rates across Illinois counties?" The question formula is:

> *"Which combination of [Candidate Predictor Variables] best explains and predicts variation in [Dependent Variable] across [Geographic/Unit Level]?"*

Keep in mind that although regression can predict outcomes, it does not establish causation. It identifies statistical associations. Two variables can be strongly correlated or predictive without one causing the other. Always interpret regression results in the context of existing theory and prior literature.

Outcome Variable vs. Predictor Variables
Before choosing a regression model, you must identify independent and dependent variables.

Outcome Variable (Y)	**Predictor Variables (X)**
Also called: dependent variable, response variable, Y variable	Also called: independent variables, explanatory variables, X variables
The **health outcome** you are trying to explain or predict.	The **SDOH factors** you believe are associated with the outcome.
Examples in this toolkit: • Diabetes prevalence (%) • Depression prevalence (%) • Premature death rate • Poor mental health days • Physical inactivity (%)	**Examples in this toolkit:** • Poverty rate (Domain 1) • % with bachelor's degree (Domain 2) • Uninsured rate (Domain 3) • EJI environmental burden (Domain 4) • Social Vulnerability Index (Domain 5)

Descriptive Statistics
Descriptive statistics offer a summary and overview of your dataset before modeling. They must be the initial step in any analysis. Standard regression models (e.g., linear, logistic) are sensitive to outliers, skewed distributions, and data entry errors. For example, if the maximum value for the poverty rate is 9,500%, you may have a data entry error. Always check the minimum, maximum, and standard deviation before fitting any model.

Statistical Metric	What Descriptive Statistics Tell Us
N (sample size)	How many counties have complete data for each variable? Missing data reduces your effective sample size.
Mean	The average value across all counties. For example, the average diabetes prevalence across all 102 Illinois counties.
Standard deviation (SD)	How spread out the values are. A high SD means counties vary widely; a low SD means they are similar to each other.
Min and Max	The most extreme values. Check for implausible values (e.g., a poverty rate of 200%) that indicate data errors.
Median	The middle value. If they differ greatly from the mean, the distribution is skewed.
Missing values	Variables with many missing values may produce unreliable model results. Investigate why the data are missing before proceeding.

Multiple Linear Regression: Concept and Interpretation
Multiple linear regression is used when your outcome variable is continuous, such as a percentage, rate, or score. It models how multiple predictor variables jointly relate to that outcome. The linear regression equation:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

Where:
- Y: The outcome variable (e.g., diabetes prevalence %)
- β_0 (intercept): The predicted value of Y when all predictors equal zero
- β_1, β_2, ... β_n (coefficients): The change in Y for a one-unit increase in each predictor, holding all other predictors constant.
- X_1, X_2, ... X_n: The predictor variables (e.g., poverty rate, education level)
- ε (error term): The residual, the part of Y not explained by the model

How to Interpret a Linear Regression Coefficient
Once you run your regression model, you must evaluate the outputs, including the direction and magnitude, statistical significance, and total model fit.

Output value	What it means	Plain-language example
Coefficient (β): Direction and Magnitude. Coefficients (β) examine variables individually. For each one-unit increase in predictor X, the outcome Y changes by β units, holding all other predictors constant. If the Coefficient (β) is...		
Positive (e.g., +0.18)	For each 1-unit increase in *X*, *Y* increases by 0.18 units. Units: X = %; Y = %	Poverty Rate (β = 0.18): Each 1 percentage point increase in county poverty rate is associated with a 0.18 percentage point higher diabetes prevalence.
Negative (e.g., -0.22)	For each 1-unit increase in *X*, *Y* decreases by 0.22 units Units: X = %; Y = %	Bachelor's Degree (β = -0.22): Each 1 percentage point increase in college education is associated with a 0.22 percentage point lower diabetes prevalence.
***p*-value and 95% Confidence Interval (CI): Statistical Significance.** Interpret the coefficient together with its confidence interval and p-value. A non-significant estimate can still be interpreted, but its uncertainty should be acknowledged.		
$p < 0.05$	Association is statistically significant at the 95% confidence level	$p = 0.003$: There is a 0.3% chance of seeing this result if the true relationship were zero.

$p > 0.05$	Association is not statistically significant	$p = 0.42$: Result is not significant; the predictor may not be associated with the outcome.
95% CI does not include 0	Range of plausible coefficient values does not cross zero. Indicates statistical significance.	CI [0.05, 0.31] is significant CI [−0.04, 0.40] is not significant (crosses zero)
R^2 (R-Squared): Total Model Fit. R^2 examines the model. It represents the percentage of variance in the outcome explained by predictors.		
R^2 (e.g., 0.65)	Percentage of variance in the outcome explained by all predictors combined	$R^2 = 0.65$: Predictors explain 65% of the county-to-county variation in diabetes prevalence; 35% is unexplained.

Controlling for Other Variables

The phrase "holding all other predictors constant" highlights the advantage of multiple regression over simple correlation. Let's think about what we know. Poverty rate and low education are strongly correlated with each other and with diabetes. A simple correlation cannot separate these effects. However, multiple regression estimates the independent contribution of each predictor after removing the shared influence of the others. This explains why a multiple regression coefficient can differ from a simple correlation.

Assumptions of Multiple Linear Regression

For linear regression results to be valid, six assumptions must be evaluated. Violating an assumption does not always mean discarding your results, but it does mean acknowledging the violation and adjusting your approach.

1

Linearity

- **What it means:** The relationship between each predictor and the outcome is a straight line.
- **Why it matters:** Linear regression fits a straight line through your data.
- **How to check:** Residuals vs Fitted plot. Points should scatter randomly around the horizontal zero line with no curve or pattern.
- **What to do if violated:** Log-transform the outcome variable (log(Y)) if it is right-skewed or add a squared term (X^2) to model the curve.

2

Independence of observations

- **What it means:** Each observation is independent; knowing the value for one geographic area does not tell you about the value for its neighbor.
- **Why it matters:** If geographic areas are spatially clustered (neighboring areas share similar health outcomes), this assumption is violated.
- **How to check:** For area-level data, this is largely guided by study design. Advanced researchers test spatial autocorrelation using Moran's I test (`sf` and `spdep` R packages).
- **What to do if violated:** Address this as a limitation in your methods section. For this toolkit's scope, standard regression is acceptable with this caveat.

3

Homoscedasticity (equal variance of residuals)

- **What it means:** The spread of residuals (prediction errors) is roughly the same across all predicted values.
- **Why it matters:** If residuals fan out (heteroscedasticity), the standard errors are incorrect, leading to wrong *p*-values and confidence intervals.
- **How to check:** Scale-Location plot (a flat red line indicates equal variance) and the Breusch-Pagan test ($p > 0.05$ indicates the assumption is met).
- **What to do if violated:** Use heteroscedasticity-robust standard errors via the `sandwich` R package: `coeftest(model, vcov = vcovHC(model))`. The coefficients remain identical; only the standard errors and *p*-value are adjusted.

4

Normality of residuals

- **What it means:** The differences between the model's predictions and the actual values (the residuals) follow a normal distribution.
- **Why it matters:** The *p*-values and CIs are calculated assuming normally distributed residuals. Linear regression is robust to mild non-normality, especially with a sample size of 100 or more, due to the central limit theorem.
- **How to check:** Normal Q-Q plot (points should fall along the diagonal line) and Shapiro-Wilk test ($p > 0.05$ indicates the assumption is met).
- **What to do if violated:** If the Q-Q plot shows heavy skew, try log-transforming the outcome variable: `lm(log(outcome) ~ predictors)`. Back-transform coefficients when reporting.

5

No multicollinearity

- **What it means:** Predictor variables are not too highly correlated with each other. It is expected and ok if they correlate with the outcome.
- **Why it matters:** When two predictors carry nearly the same information, the model cannot reliably estimate each predictor's contribution. Coefficients become unstable, and standard errors inflate dramatically. Including both poverty rate and SVI socioeconomic score causes the model to explain poverty twice, making estimates unreliable.
- **How to check:** Variance Inflation Factor (VIF). VIF < 5 is acceptable. VIF 5–10 is a moderate concern. VIF > 10 is a serious problem, and the predictor needs to be removed.
- **What to do if violated:** Remove the predictor with the highest VIF and re-run the model. Repeat until all VIF < 5.

6

No highly influential outliers

- **What it means:** No single geographic area is excessively driving the regression results on its own.
- **Why it matters:** An outlier geographic area with unusual traits, like high poverty but low diabetes rates, can significantly influence the regression line. Your conclusion should reflect overall geographic patterns, not be dictated by one or two anomalies.
- **How to check:** Cook's Distance. A common threshold is 4/n (where n is the number of geographic areas). Geographic areas above this threshold are flagged as potentially influential.

$$D_i = \frac{e_i^2}{p \cdot MSE} \times \frac{h_{ii}}{(1 - h_{ii})^2}$$

- **What to do if violated:** Run the model with and without the flagged areas. If the results change substantially, report both models and discuss the influential observation in your limitations section.

Logistic Regression: Concept and Interpretation

Logistic regression is used when your outcome variable is binary, such as yes/no, high/low, or above threshold/below threshold. You cannot use linear regression for a binary outcome because it can predict values outside the 0–1 range.

The logistic regression model

Instead of predicting the outcome value, logistic regression predicts the probability that the outcome equals 1. For example, the probability that a county has a high diabetes rate.

$$log(\frac{P}{1 - P}) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

The left side is the log-odds (also called the logit) of the outcome occurring. The right side is the same linear combination of predictors in linear regression. Coefficients are estimated in log-odds units, then converted to Odds Ratios (OR) for interpretation.

What is an Odds Ratio?

The Odds Ratio (OR) is the output of logistic regression. It measures the change in the odds of the outcome for a one-unit increase in the predictor, holding all other variables constant. It is calculated by exponentiating the raw model coefficient:

$$OR = e^{\beta}$$

Odds Ratio Interpretation Table

Outcome Variable (*Y*): Whether a county falls into the top 50% of diabetes prevalence in the state (1 = Yes, 0 = No).

Output Metric	What it Means	Plain-Language Example
OR = 1.0	No association	Voter turnout (OR = 1.0): No association with the odds of the county falling into the top half of diabetes prevalence.
OR > 1.0 (e.g., 1.25)	Higher predictor value is associated with higher odds of the outcome	Poverty rate (OR = 1.25): Each 1 percentage point increase in county poverty rate is associated with 25% higher odds of the county falling into the top half of diabetes prevalence.
OR < 1.0 (e.g., 0.78)	Higher predictor value is associated with lower odds of the outcome.	Bachelor's degree (OR = 0.78): Each 1 percentage point increase in college graduation is associated with 22% lower odds of the county falling into the top half of diabetes prevalence.
$p < 0.05$	Statistically significant association	$p = 0.01$: There is only a 1% chance of observing this result if the true population association were zero.
95% CI excludes 1.0	Another way to confirm significance	CI [1.05, 1.52] does not include 1.0 → significant; CI [0.95, 1.38] includes 1.0 → not significant

How the Binary Outcome is Created in This Toolkit

For continuous outcome variables (percentages, rates, or scores), the script creates a binary outcome by comparing each geographic area to the state median. Variables that are already binary, such as `hpsa_any_shortage`, can be used as the outcome without this conversion step.

```
# Areas above the state median diabetes rate → high diabetes = 1
# Areas at or below the state median        → high_diabetes = 0
# Example for Illinois (median diabetes ≈ 11.5%):
# Cook County    diabetes = 10.2%  →  high diabetes = 0 (below median)
# Alexander County diabetes = 17.8% →  high_diabetes = 1 (above median)
```

Assumptions of Logistic Regression
Logistic regression has fewer distributional assumptions than linear regression, but it has its own requirements that must be checked.

1

Binary outcome

- **What it means:** The outcome must only have two values, 0 or 1. Logistic regression requires a binary outcome. Continuous outcomes should ordinarily be analyzed using an appropriate continuous-outcome model. In this toolkit, median dichotomization is used as a teaching demonstration when a naturally binary outcome is unavailable. For example, coding an outcome as 1 if it is above the median and 0 if it is below.
- **How to check:** Verify your outcome variable containing two unique values using R: `table(logistic_df$outcome_high)`

2

Independence of observations

- **What it means:** Each geographic unit must be an independent observation.
- **How to check:** Review the study design. Advanced analyses, such as spatial autocorrelation, may be necessary. If not, this should be stated in the limitations section or addressed through spatial analyses (outside this handout's scope).

3

No multicollinearity

- **What it means:** Predictor variables should not be too highly correlated with each other. For example, both poverty and an overall SVI might cause multicollinearity because SVI is partly derived from ACS poverty data.
- **How to check:** Variance Inflation Factor (VIF). Just like linear regression, a VIF < 5 is acceptable for all predictors.

4

No complete or quasi-complete separation

- **What it means:** Separation occurs when predictors perfectly split the binary outcome, predicting whether an observation is 0 or 1. This can cause unstable model estimates, extremely large odds ratios, and inflated standard errors. It is more likely with small samples, rare outcomes, or threshold-based outcomes.
- **How to check:** Look for very large coefficients that are greater than 10 on the log-odds scale in your R output.

5

Adequate sample size (events per variable)

- **What it means:** Logistic regression needs enough counties with outcome = 1 and outcome = 0 for each predictor. A common rule of thumb is at least 10 events per predictor variable. For example, with about 51 high-diabetes counties and 7 predictors, 51 / 7 ≈ 7 events per predictor, which is borderline. To improve model stability, use 5 or fewer predictors.
- **How to check:** Calculates EPV = `n_events / length(PREDICTOR_VARS).`

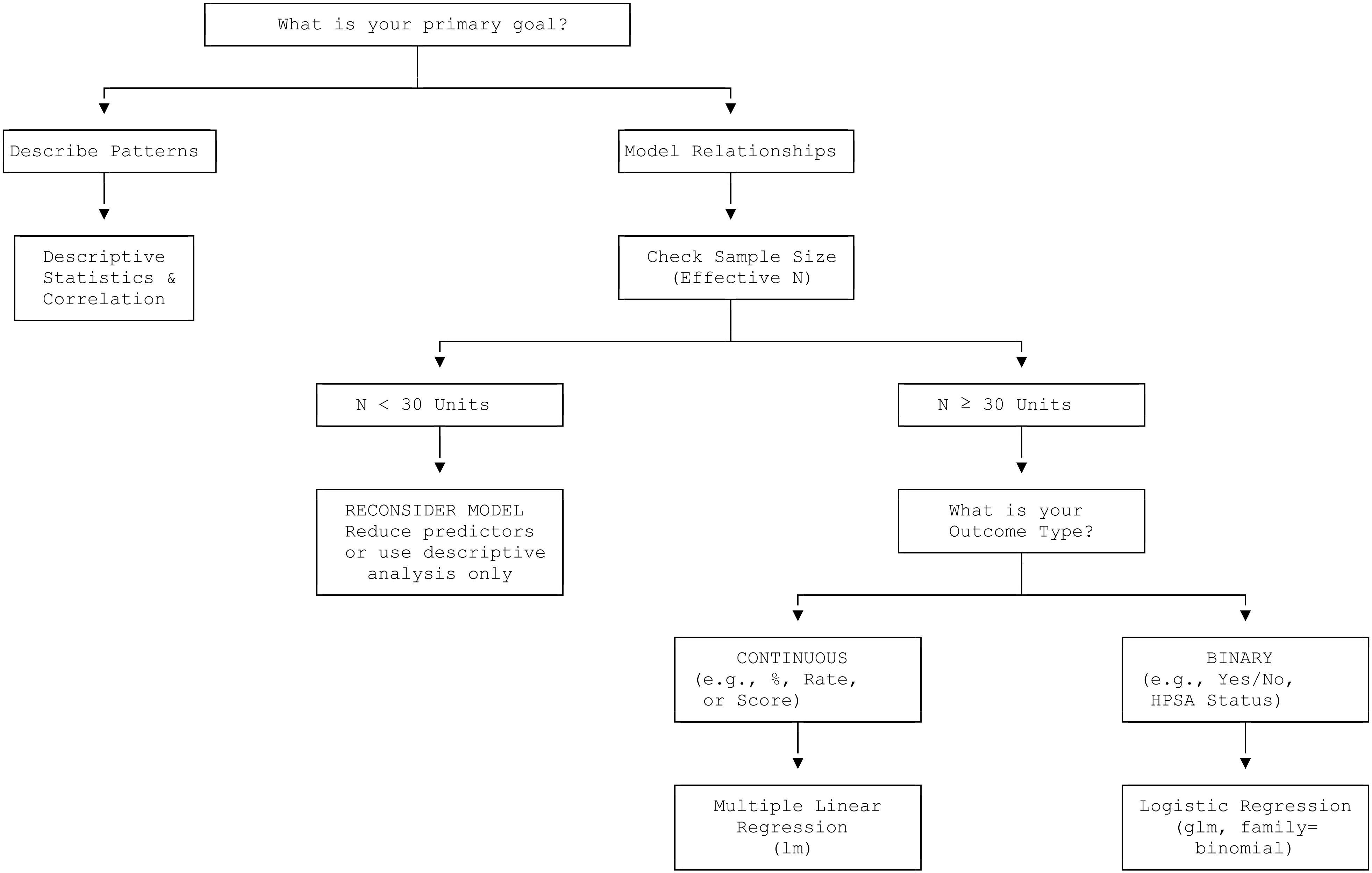
6

Good model fit

- **What it means:** The model's predicted probabilities should match the observed outcomes adequately. In other words, the model should be a good fit for the data. If not, predicted probabilities will be inaccurate and misleading.
- **How to check:** Hosmer-Lemeshow test. $p > 0.05$ indicates adequate fit. $p < 0.05$ means the model may need revision.

Quick guide: which regression should I use?

- My outcome is a percentage, rate, or score (e.g., diabetes %, premature death rate) → **Use multiple linear regression (lm)**
- My outcome is yes/no, above/below a threshold, or a binary designation (e.g., HPSA designated county) → **Use logistic regression (glm, family=binomial)**
- I want to describe patterns without making causal claims → **Use both: descriptive statistics and correlation matrix first, then regression**
- I have fewer than 30 counties after removing missing values → **Reconsider your model — reduce the number of predictors or consider descriptive analysis only**
- I want to compare one outcome across all five HP 2030 domains simultaneously → **Run separate models for each domain pair, or run one fully adjusted model with one predictor per domain**


What is your primary goal?
Describe Patterns
Model Relationships
Descriptive Statistics & Correlation
Check Sample Size (Effective N)
N < 30 Units
N ≥ 30 Units
RECONSIDER MODEL Reduce predictors or use descriptive analysis only
What is your Outcome Type?
CONTINUOUS (e.g., %, Rate, or Score)
BINARY (e.g., Yes/No, HPSA Status)
Multiple Linear Regression (lm)
Logistic Regression (glm, family= binomial)

Flexible Variable Selection
All analyses in Script #7 are read from a single configuration block at the top of the script. You only need to edit this one block to change your outcome variable, your predictors, or both. The rest of the script runs automatically with your chosen variables.

How To Choose Your Variables

- Outcome variable (*Y*) must be the variable you are trying to explain or predict.
  - For linear regression, it must be continuous (e.g., `diabetes_pct`, `depression_pct`).
  - For logistic regression, continuous outcomes will be converted to binary inside the script.
- Predictor variables (*X*): the SDOH factors you believe are associated with your outcome. Choose variables from different domains to test a multi-domain model. Avoid using two variables that measure nearly the same thing (e.g., `poverty_rate` AND `pct_poverty_svi` together).
  - For linear regression, predictors must be continuous.
  - For logistic regression, predictors can be continuous, categorical, or a mix of both.

Rule of thumb for sample size: you need at least 10-20 observations per predictor variable. For example, at the county level, Illinois has 102 observations, which comfortably supports approximately 5-7 predictor variables. Adjust this rule based on your chosen state and geographic unit. See the variable reference table for the full list of available variables and their domains.

```
# ════════════════════════════════════════════════════════════════════════════
# Script #7: Statistical Analyses
# ════════════════════════════════════════════════════════════════════════════
# EDIT THIS BLOCK ONLY — change your outcome and predictor variables here.
# Everything else in the script uses these names automatically.
# ════════════════════════════════════════════════════════════════════════════

library(tidyverse)
library(skimr)
library(corrplot)
library(car)
library(broom)
library(lmtest)   # Breusch-Pagan test for homoscedasticity
library(ResourceSelection)  # Hosmer-Lemeshow test for logistic regression

# ── Load master dataset ──────────────────────────────────────────────────────
STATE LABEL <- "il"   # must match what you set in Script 06
master <- read csv(paste0("data/master/sdoh ", STATE LABEL, " master.csv"))

# ════════════════════════════════════════════════════════════════════════════
# VARIABLE CONFIGURATION — EDIT THIS SECTION TO CHANGE YOUR ANALYSIS
# ════════════════════════════════════════════════════════════════════════════

# ── Step 1: Choose your OUTCOME variable (Y) ────────────────────────────────
# Must be a continuous variable for linear regression.
# It will be dichotomized automatically for logistic regression.
#
# Available outcome options (change the value in quotes below):
#   "diabetes_pct"        — Diagnosed diabetes prevalence (%)
#   "depression pct"      — Depression prevalence (%)
#   "obesity_pct"         — Adult obesity prevalence (%)
#   "hypertension pct"    — High blood pressure prevalence (%)
#   "smoking pct"         — Current smoking prevalence (%)
#   "inactivity_pct"      — Physical inactivity prevalence (%)
```

```
#   "premature_death"     — Premature death rate (YPLL per 100k)
#   "poor_mental_days"    — Avg mentally unhealthy days/month
#   "poor_health_days"    — Avg physically unhealthy days/month
#
OUTCOME_VAR <- "diabetes_pct"   # <—— CHANGE THIS

# ── Step 2: Choose your PREDICTOR variables (X) ──────────────────────────────
# List 3-7 variables. More than 7 is too many for n=102 counties.
#
# Available predictor options:
#  -- Domain 1: Economic Stability --
#   "poverty_rate"        — % persons below poverty line
#   "unemployment_rate"   — % civilian labor force unemployed
#   "median_hh_income"    — median household income ($)
#   "snap_rate"           — % households receiving SNAP
#   "rent_burden_rate"    — % renters paying 30%+ income on rent
#
#  -- Domain 2: Education --
#   "pct_no_hs"           — % adults 25+ with less than HS diploma
#   "pct_bachelor_plus"   — % adults 25+ with bachelor's degree or higher
#   "pct_free_lunch"      — % students eligible for free lunch (NCES CCD)
#   "total_enrollment"    — total student enrollment in county (NCES CCD)
#
#  -- Domain 3: Health Care Access --
#   "uninsured_rate"            — % population uninsured (ACS)
#   "hpsa_prim_care"            — primary care shortage (0=none, 1=partial, 2=full)
#   "hpsa_dental"               — dental shortage (0=none, 1=partial, 2=full)
#   "hpsa_mental_health"       — mental health shortage (0=none, 1=partial, 2=full)
#   "hpsa_any_shortage"         — any shortage designation (1=yes, 0=no)
#   "hpsa_prim_care_designated" — primary care shortage binary (1=yes, 0=no)
#
#  -- Domain 4: Built Environment --
#   "pct_lila_tracts"     — % tracts that are low-income & low-access food
#   "eji_overall"         — EJI overall environmental burden percentile
#   "eji_env_burden"      — EJI environmental burden percentile
#
#  -- Domain 5: Social & Community Context --
#   "svi_overall"         — SVI overall vulnerability percentile (0-1)
#   "svi_socioeco"        — SVI Theme 1: socioeconomic status
#   "svi_minority"        — SVI Theme 3: minority status & language (as defined by
#                                     CDC/ATSDR SVI documentation)
#   "social_associations" — membership organizations per 100k
#   "voter_turnout"       — % voting-age pop who voted
#
PREDICTOR_VARS <- c(
  "poverty_rate",         # <—— EDIT THIS LIST
  "pct_bachelor_plus",
  "uninsured_rate",
  "pct_lila_tracts",
  "social_associations"
)

# ── Step 3 (optional): Add a label for your output files ─────────────────────
ANALYSIS_LABEL <- "diabetes_sdoh_model"   # <—— CHANGE TO DESCRIBE YOUR MODEL

# ── Build analysis dataset from your chosen variables ─────────────────────────
all_vars   <- c("fips", "county_name", OUTCOME_VAR, PREDICTOR_VARS)
```

```
analysis df <- master %>%
  select(all of(all vars)) %>%
  drop_na()

cat("\n── Analysis configuration ──────────────────────────────\n")
cat("Outcome:    ", OUTCOME_VAR, "\n")
cat("Predictors: ", paste(PREDICTOR VARS, collapse=", "), "\n")
cat("Sample size:", nrow(analysis df), "observations (after removing missing)\n")
cat("Counties dropped due to missing:", nrow(master) - nrow(analysis_df), "\n")
```

Variable Reference Table

Use this table to choose your outcome and predictor variables. All variable names listed here are exact column names in the master CSV file.

Variable name	Label	Domain	Type	Source
diabetes_pct	Diagnosed diabetes (%)	Health outcome	Continuous	CDC PLACES 2025
depression_pct	Depression (%)	Health outcome	Continuous	CDC PLACES 2025
obesity_pct	Adult obesity (%)	Health outcome	Continuous	CDC PLACES 2025
hypertension_pct	High blood pressure (%)	Health outcome	Continuous	CDC PLACES 2025
smoking_pct	Current smoking (%)	Health outcome	Continuous	CDC PLACES 2025
inactivity_pct	Physical inactivity (%)	Health outcome	Continuous	CDC PLACES 2025
premature_death	Premature death rate (YPLL/100k)	Health outcome	Continuous	CHR&R 2025
poor_mental_days	Mentally unhealthy days/mo	Health outcome	Continuous	CHR&R 2025
poor_health_days	Physically unhealthy days/mo	Health outcome	Continuous	CHR&R 2025
poverty_rate	% below poverty line	Domain 1	Continuous	ACS 2020-2024
unemployment_rate	% unemployed	Domain 1	Continuous	ACS 2020-2024
median_hh_income	Median household income ($)	Domain 1	Continuous	ACS 2020-2024
snap_rate	% HH receiving SNAP	Domain 1	Continuous	ACS 2020-2024
rent_burden_rate	% renters cost-burdened 30%+	Domain 1	Continuous	ACS 2020-2024
pct_no_hs	% adults <HS diploma	Domain 2	Continuous	ACS 2020-2024
pct_bachelor_plus	% adults bachelor's degree+	Domain 2	Continuous	ACS 2020-2024
n_districts	Number of school districts	Domain 2	Count	NCES CCD 2024
total_enrollment	Total student enrollment	Domain 2	Count	NCES CCD 2024
pct_free_lunch	% students free lunch eligible	Domain 2	Continuous	NCES CCD 2024
pct_reduced_lunch	% students reduced lunch eligible	Domain 2	Continuous	NCES CCD 2024

uninsured_rate	% population uninsured	Domain 3	Continuous	ACS 2020-2024
hpsa_prim_care	Primary care HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Ordinal	AHRF 2025
hpsa_dental	Dental HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Ordinal	AHRF 2025
hpsa_mental_health	Dental HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Ordinal	AHRF 2025
hpsa_any_shortage	Dental HPSA designation (0=none, 1=partial, 2=full)	Domain 3	Binary	AHRF 2025
hpsa_prim_care_designated	Primary care shortage (1=yes, 0=no)	Domain 3	Binary	AHRF 2025
pct_lila_tracts	% low-income & low-access tracts	Domain 4	Continuous	USDA 2019
eji_overall	EJI overall percentile (0-1)	Domain 4	Continuous	EJI 2024
eji_env_burden	EJI environmental burden	Domain 4	Continuous	EJI 2024
svi_overall	SVI overall percentile (0-1)	Domain 5	Continuous	SVI 2022
svi_socioeco	SVI Theme 1: socioeconomic	Domain 5	Continuous	SVI 2022
svi_minority	SVI Theme 3: minority & language	Domain 5	Continuous	SVI 2022
social_associations	Social associations per 100k	Domain 5	Continuous	CHR&R 2025
voter_turnout	Voter turnout %	Domain 5	Continuous	CHR&R 2025

Avoid Collinear Predictors

- Do not include both `svi_overall` and `poverty_rate` in the same model. They are strongly correlated (SVI is partly derived from ACS poverty data). Similarly, avoid combining highly overlapping measures without first reviewing correlations and VIFs.
- Run the correlation matrix first. Treat an absolute correlation above about .70 as a warning. Review correlations, VIFs, substantive knowledge, and the research question before deciding whether to remove a predictor.

Descriptive Statistics Table

Always start with descriptive statistics. This gives you a complete picture of the distribution, missing values, and range of every variable before running any models.

```
# ══ PART C: Descriptive Statistics ══════════════════════════════════════════
# Uses OUTCOME VAR and PREDICTOR VARS defined in the configuration block above.

# Full skim of all key variables
skim output <- analysis df %>%
```

```
  select(all_of(c(OUTCOME_VAR, PREDICTOR_VARS))) %>%
  skim()
print(skim_output)

# ── Formatted summary table ──────────────────────────────────────────
desc_stats <- analysis_df %>%
  select(all_of(c(OUTCOME_VAR, PREDICTOR_VARS))) %>%
  pivot_longer(everything(), names_to="variable", values_to="value") %>%
  group_by(variable) %>%
  summarise(
    n       = sum(!is.na(value)),
    missing = sum(is.na(value)),
    mean    = round(mean(value,   na.rm=TRUE), 2),
    sd      = round(sd(value,     na.rm=TRUE), 2),
    min     = round(min(value,    na.rm=TRUE), 2),
    median  = round(median(value, na.rm=TRUE), 2),
    max     = round(max(value,    na.rm=TRUE), 2)
  )

print(desc_stats)
write_csv(desc_stats,
  paste0("output/tables/01_descriptive_", ANALYSIS_LABEL, ".csv"))
```

Correlation Matrix

Run the correlation matrix before regression. It reveals which predictors are strongly associated with the outcome AND with each other. High correlations between predictors ($r > 0.70$) indicate multicollinearity and require dropping one of the correlated variables.

```
# ══ PART D: Correlation Matrix ══════════════════════════════════════

cor_data <- analysis_df %>%
  select(all_of(c(OUTCOME_VAR, PREDICTOR_VARS))) %>%
  drop_na()

cor_matrix <- cor(cor_data, method="pearson")

# ── Plot ─────────────────────────────────────────────────────────────
corrplot(
  cor_matrix,
  method      = "color",
  type        = "upper",
  order       = "hclust",
  addCoef.col = "black",
  tl.col      = "black",
  tl.srt      = 45,
  number.cex  = 0.75,
  title       = paste("Correlation Matrix —", OUTCOME_VAR),
  mar         = c(0,0,2,0)
)

# Save plot
png(paste0("output/figures/02_correlation_", ANALYSIS_LABEL, ".png"),
    width=1400, height=1200, res=150)
corrplot(cor_matrix, method="color", type="upper", order="hclust",
         addCoef.col="black", tl.col="black", tl.srt=45, number.cex=0.75)
dev.off()
```

```
# ── Flag high inter-predictor correlations ────────────────────────────────────
pred_cor <- cor(analysis_df %>% select(all_of(PREDICTOR_VARS)), method="pearson")
high_pairs <- which(abs(pred_cor) > 0.70 & upper.tri(pred_cor), arr.ind=TRUE)

if (nrow(high_pairs) > 0) {
  cat("\nWARNING — High correlations (r > 0.70) between predictors:\n")
  for (i in seq_len(nrow(high_pairs))) {
    r1 <- rownames(pred_cor)[high_pairs[i,1]]
    r2 <- colnames(pred_cor)[high_pairs[i,2]]
    r  <- round(pred_cor[high_pairs[i,1], high_pairs[i,2]], 2)
    cat(" ", r1, "vs", r2, ":", r, "\n")
  }
  cat("ACTION: Consider removing one variable from each highly correlated pair.\n")
} else {
  cat("\nNo high inter-predictor correlations (all r < 0.70). Proceed.\n")
}
```

Why We Run Both Unadjusted and Adjusted Models

This toolkit runs both unadjusted and adjusted models for linear and logistic regression. For logistic regression, presenting unadjusted (crude) and adjusted odds ratios side by side is a well-established standard in epidemiological reporting. Bagley et al. (2001) in the Journal of Clinical Epidemiology recommend that both unadjusted and adjusted estimates always be presented so readers can assess the degree of confounding. For linear regression, although reporting practices vary across the health research literature, presenting unadjusted coefficients alongside adjusted coefficients serves an important pedagogical and analytical purpose. It shows how much each predictor's association with the outcome changes after controlling for other variables, which captures the concept of confounding in SDOH research. For each predictor, an unadjusted model is run separately with only that predictor and the outcome. Then, a fully adjusted model is run with all predictors together. Comparing the two reveals which predictors are independently associated with the outcome, and which associations are explained by other SDOH factors.

References

- Bagley, S. C., White, H., & Golomb, B. A. (2001). Logistic regression in the medical literature: Standards for use and reporting, with particular attention to one medical domain. *Journal of Clinical Epidemiology*, 54(10), 979-985. https://doi.org/10.1016/S0895-4356(01)00372-9
- Jones, L., Barnett, A., & Vagenas, D. (2025). Linear regression reporting practices for health researchers, a cross-sectional meta-research study. *PLOS ONE*. https://doi.org/10.1371/journal.pone.0305150
- Sperandei, S. (2014). Understanding logistic regression analysis. *Biochemia Medica*, 24(1), 12-18. https://doi.org/10.11613/BM.2014.003

Multiple Linear Regression with Full Assumption Checks

Multiple linear regression requires five assumptions to hold for the results to be valid. This section fits the model and systematically checks each assumption using both statistical tests and diagnostic plots.

Five Assumptions of Multiple Linear Regression

1. **Linearity —** Each predictor has a linear relationship with the outcome. Check: residuals vs fitted plot. Points should scatter randomly around zero with no curve.
2. **Independence —** Observations are independent of each other. Check: confirmed by study design. Each observation is a separate unit.
3. **Homoscedasticity —** Residual variance is constant across fitted values. Check: Scale-Location plot & Breusch-Pagan test. Violation: use robust standard errors.

4. **Normality of residuals —** Residuals follow a normal distribution. Check: Q-Q plot & Shapiro-Wilk test.
5. **No influential outliers —** No single county is excessively driving the results. Check: Cook's Distance plot. Observations with Cook's D > 4/n are flagged for review.

```
# ═══ PART E: Multiple Linear Regression + Assumption Checks ══════════════════

# ─── Unadjusted models — one per predictor ─────────────────────────────────────
# Standard epidemiological practice: run one unadjusted model per predictor
# Each gives a crude regression coefficient (unadjusted β)
cat("\n─── Unadjusted linear models (crude β — one predictor each) ───\n")

unadj lm results <- map dfr(PREDICTOR VARS, function(var) {
  formula_unadj <- as.formula(paste(OUTCOME_VAR, "~", var))
  model unadj   <- lm(formula unadj, data = analysis df)
  tidy(model unadj, conf.int = TRUE) %>%
    filter(term == var) %>%
    mutate(model = "Unadjusted", predictor = var,
           across(where(is.numeric), ~round(., 4)))
})

print(unadj lm results %>%
        select(predictor, estimate, conf.low, conf.high, p.value))

# ─── Fully adjusted model — all predictors together ────────────────────────────
cat("\n─── Fully adjusted model (all predictors) ───\n")
formula_adj <- as.formula(paste(OUTCOME_VAR, "~",
                                paste(PREDICTOR VARS, collapse=" + ")))
model adj   <- lm(formula adj, data = analysis df)
summary(model adj)

# ═════════════════════════════════════════════════════════════════════════════
# ASSUMPTION CHECKS — MULTIPLE LINEAR REGRESSION
# ═════════════════════════════════════════════════════════════════════════════

cat("\n══ ASSUMPTION CHECKS: MULTIPLE LINEAR REGRESSION ══\n")

# ── Assumption 1 & 3 & 5: Diagnostic plots ─────────────────────────────────────
# Four plots: (1) Residuals vs Fitted, (2) Normal Q-Q,
#             (3) Scale-Location,      (4) Residuals vs Leverage
png(paste0("output/figures/03 lm diagnostics ", ANALYSIS LABEL, ".png"),
    width=1600, height=1400, res=150)
par(mfrow=c(2,2))
plot(model adj, main=paste("LM Diagnostics —", OUTCOME VAR))
par(mfrow=c(1,1))
dev.off()
cat("Diagnostic plots saved to output/figures/\n")
cat("Interpret: (1) Residuals vs Fitted — no pattern = linearity + homoscedasticity
OK\n")
cat("           (2) Q-Q plot — points on line = normality OK\n")
cat("           (3) Scale-Location — flat red line = homoscedasticity OK\n")
cat("           (4) Cook's Distance — no points > dashed line = no influential
outliers\n")

# ── Assumption 3: Breusch-Pagan test for homoscedasticity ──────────────────────
cat("\n── Assumption 3: Homoscedasticity (Breusch-Pagan test) ──\n")
```

```
bp_test <- bptest(model_adj)
print(bp_test)
if (bp_test$p.value < 0.05) {
  cat("RESULT: p <0.05 — heteroscedasticity detected (unequal variance).\n")
  cat("ACTION: Use heteroscedasticity-consistent (robust) standard errors.\n")
  cat("        Run: library(sandwich); coeftest(model_adj,
vcov=vcovHC(model_adj))\n")
} else {
  cat("RESULT: p >", round(bp_test$p.value,3), "— homoscedasticity assumption
met.\n")
}

# —— Assumption 4: Shapiro-Wilk test for normality of residuals ———————————————
cat("\n—— Assumption 4: Normality of residuals (Shapiro-Wilk test) ——\n")
sw_test <- shapiro.test(residuals(model_adj))
print(sw_test)
if (sw_test$p.value < 0.05) {
  cat("RESULT: p <0.05 — residuals may not be normally distributed.\n")
  cat("NOTE:   With n=102, linear regression is generally robust to mild\n")
  cat("        non-normality. Check the Q-Q plot visually.\n")
  cat("        If Q-Q plot shows heavy skew, consider log-transforming the
outcome.\n")
} else {
  cat("RESULT: p >", round(sw_test$p.value,3), "— normality assumption met.\n")
}

# —— Assumption 5: Cook's Distance — influential observations —————————————————
cat("\n—— Assumption 5: Influential observations (Cook's Distance) ——\n")
cooks_d    <- cooks.distance(model_adj)
cutoff     <- 4 / nrow(analysis_df)   # common threshold: 4/n
influential <- which(cooks_d > cutoff)
cat("Cook's D threshold (4/n):", round(cutoff, 4), "\n")
if (length(influential) > 0) {
  cat("Influential observations (Cook's D >", round(cutoff,4), "):\n")
  print(analysis_df[influential, c("fips","county_name", OUTCOME_VAR)])
  cat("ACTION: Run sensitivity analyses - see Follow-up 2 below.\n")
} else {
  cat("No influential outliers detected.\n")
}

# —— Multicollinearity: Variance Inflation Factor (VIF) ———————————————————————
cat("\n—— Multicollinearity check: Variance Inflation Factor (VIF) ——\n")
cat("VIF < 5 = acceptable | VIF 5-10 = moderate concern | VIF > 10 = serious
problem\n")
vif_vals <- vif(model_adj)
print(round(vif_vals, 2))
if (any(vif_vals > 5)) {
  cat("WARNING: VIF > 5 detected. Consider removing correlated predictors.\n")
  cat("High VIF variables:", names(which(vif_vals > 5)), "\n")
} else {
  cat("VIF OK — no multicollinearity problem detected.\n")
}

# —— Model results table ——————————————————————————————————————————————————————
cat("\n—— Final regression results ——\n")
results_lm <- tidy(model_adj, conf.int=TRUE) %>%
  mutate(across(where(is.numeric), ~round(.,4)))
```

```
print(results_lm)

fit_lm <- glance(model_adj)
cat("R² =", round(fit_lm$r.squared,3),
    " | Adj. R² =", round(fit_lm$adj.r.squared,3),
    " | F-stat p =", round(fit_lm$p.value,4), "\n")

write_csv(results_lm,
  paste0("output/tables/03_lm_results_", ANALYSIS_LABEL, ".csv"))
```

Follow-Up Analyses Based On The Assumption Check Results:

```
# ════════════════════════════════════════════════════════════════════════════════
# FOLLOW-UP ANALYSES BASED ON ASSUMPTION CHECKS
# ════════════════════════════════════════════════════════════════════════════════

library(sandwich)  # robust standard errors

# ─── Follow-up 1: Robust standard errors (required if Breusch-Pagan p < 0.05) ───
if (bp_test$p.value < 0.05) {
  cat("\n─── Robust standard errors (heteroscedasticity correction) ───\n")
  library(sandwich)
  robust_results <- coeftest(model_adj, vcov = vcovHC(model_adj))
  print(robust_results)
  cat("NOTE: Use these standard errors and p-values for reporting.\n")
  cat("      Coefficients are identical — only standard errors change.\n")
}

# ─── Follow-up 2: Sensitivity analysis (remove influential observations) ────────
if (length(influential) > 0) {
  cat("\n─── Sensitivity analysis (removing influential observations) ───\n")

  # Use the influential observations flagged by Cook's Distance above
  influential_fips <- analysis_df$fips[influential]
  cat("Removing observations with FIPS:", influential_fips, "\n")

  analysis_df_trim <- analysis_df %>%
    filter(!fips %in% influential_fips)

  model_trim <- lm(formula_adj, data = analysis_df_trim)

  cat("Trimmed model (n =", nrow(analysis_df_trim), "):\n")
  summary(model_trim)

  cat("\nComparison: key coefficients:\n")
  cat("                    Full model    Trimmed model\n")
  for (v in PREDICTOR_VARS) {
    full_coef  <- round(coef(model_adj)[v], 3)
    trim_coef  <- round(coef(model_trim)[v], 3)
    cat(sprintf("%-20s %10s    %10s\n", v, full_coef, trim_coef))
  }
  cat("\nIf coefficients are similar, results are robust to influential
observations.\n")
}
```

Logistic Regression with Full Assumption Checks

Logistic regression models the probability of a binary outcome. This script helps convert your continuous outcome variable to binary (above/below the state median), fit a logistic model, check all key assumptions, and report odds ratios.

Logistic Regression Assumptions and Diagnostics

1. **Binary outcome.** The outcome is 0/1. This script creates the binary variable automatically by dichotomizing your chosen outcome at the state median.
2. **Independence.** Observations are independent, confirmed by the study design or stated limitation in your report.
3. **No multicollinearity.** Predictors are not highly correlated with each other. Check: VIF.
4. **No complete separation.** No single predictor perfectly predicts the outcome (all 0s or all 1s for one group). This causes model failure. Check: examine frequency tables.
5. **Adequate sample size.** At least 10 events (outcome = 1) per predictor variable.
6. **Goodness of fit.** The model adequately fits the observed data. Check: Hosmer-Lemeshow test ($p >$ 0.05 means good fit).

```
# ══ PART F: Logistic Regression & Assumption Checks ═════════════════════════
library(ResourceSelection)  # Hosmer-Lemeshow test

# ── Create binary outcome at the state median ────────────────────────────────────
state_median <- median(analysis_df[[OUTCOME_VAR]], na.rm=TRUE)
binary_outcome <- paste0(OUTCOME_VAR, "_high")

logistic_df <- analysis_df %>%
  mutate(
    !!binary_outcome := as.integer(.data[[OUTCOME_VAR]] > state_median)
  )

n_events   <- sum(logistic_df[[binary_outcome]] == 1, na.rm=TRUE)
n_nonevent <- sum(logistic_df[[binary_outcome]] == 0, na.rm=TRUE)

cat("\nBinary outcome:", binary_outcome, "\n")
cat("  Threshold (state median):", round(state_median, 2),
    "[", OUTCOME_VAR, "]\n")
cat("  High (", binary_outcome, "= 1):", n_events, "observations\n")
cat("  Low  (", binary_outcome, "= 0):", n_nonevent, "observations\n")

# ── Assumption 5 check: events-per-variable ──────────────────────────────────────
cat("\n── Assumption 5: Sample size adequacy ──\n")
epv <- n_events / length(PREDICTOR_VARS)
cat("Events per predictor variable (EPV):", round(epv,1), "\n")
if (epv < 10) {
  cat("WARNING: EPV <10. Consider reducing predictor count to",
      floor(n_events/10), "or fewer.\n")
} else {
  cat("EPV OK (>=10). Sample size is adequate.\n")
}

# ── Unadjusted models — one per predictor ────────────────────────────────────────
# Standard epidemiological practice: run one unadjusted model per predictor
# Each gives a crude odds ratio (unadjusted OR)
cat("\n── Unadjusted logistic models (crude OR — one predictor each) ──\n")

unadj_results <- map_dfr(PREDICTOR_VARS, function(var) {
  formula_unadj <- as.formula(paste(binary_outcome, "~", var))
```

```
  model unadj   <- glm(formula unadj, data = logistic df,
                       family = binomial(link = "logit"))
  tidy(model_unadj, conf.int = TRUE, exponentiate = TRUE) %>%
    filter(term == var) %>%
    mutate(model = "Unadjusted", predictor = var,
           across(where(is.numeric), ~round(., 4)))
})

print(unadj_results %>%
        select(predictor, estimate, conf.low, conf.high, p.value))

# ── Fully adjusted logistic model — all predictors together ──────────────────
cat("\n── Fully adjusted logistic model (all predictors) ──\n")
formula logit <- as.formula(
  paste(binary outcome, "~", paste(PREDICTOR VARS, collapse=" + "))
)
logit model <- glm(formula logit, data=logistic df,
                   family=binomial(link="logit"))
summary(logit_model)

# ════════════════════════════════════════════════════════════════════════════
# ASSUMPTION CHECKS — LOGISTIC REGRESSION
# ════════════════════════════════════════════════════════════════════════════

cat("\n══ ASSUMPTION CHECKS: LOGISTIC REGRESSION ══\n")

# ── Assumption 3: VIF — multicollinearity ─────────────────────────────────────
cat("\n── Assumption 3: Multicollinearity (VIF) ──\n")
vif logit <- vif(logit model)
print(round(vif_logit, 2))
if (any(vif logit > 5)) {
  cat("WARNING: VIF > 5 — multicollinearity present. Remove correlated
predictors.\n")
} else {
  cat("VIF OK — no multicollinearity problem.\n")
}

# ── Assumption 4: Complete separation check ──────────────────────────────────
cat("\n── Assumption 4: Complete separation check ──\n")
# If any predictor perfectly predicts the outcome, glm() will warn about
# 'fitted probabilities numerically 0 or 1'.
# Also check for very large coefficients (> 10 in log-odds scale).
coef_check <- coef(logit_model)[-1]   # exclude intercept
if (any(abs(coef check) > 10, na.rm=TRUE)) {
  cat("WARNING: Very large coefficients detected — possible complete
separation.\n")
  cat("Variables with large coefficients:", names(which(abs(coef check)>10)), "\n")
  cat("ACTION: Check frequency table for that predictor by outcome category.\n")
} else {
  cat("No evidence of complete separation (all log-odds coefficients < 10).\n")
}

# ── Assumption 6: Hosmer-Lemeshow goodness-of-fit test ────────────────────────
cat("\n── Assumption 6: Hosmer-Lemeshow goodness-of-fit test ──\n")
cat("Null hypothesis: model fits the data adequately (p > 0.05 = good fit)\n")
hl test <- hoslem.test(
  x = logistic df[[binary outcome]],
```

```
  y = fitted(logit model),
  g = 10   # 10 groups (default)
)
print(hl test)
if (hl test$p.value < 0.05) {
  cat("RESULT: p <0.05 — model may not fit well. Consider revising predictors.\n")
} else {
  cat("RESULT: p =", round(hl test$p.value,3), "— model fits the data 
adequately.\n")
}

# —— Model fit statistics ——————————————————————————————————————————————————————
cat("\n—— Model fit statistics ——\n")
cat("AIC:", round(AIC(logit model),2), "\n")
cat("Null deviance:",     round(logit model$null.deviance,2), "\n")
cat("Residual deviance:", round(logit_model$deviance,2), "\n")
mcfadden <- 1 - (logit model$deviance / logit model$null.deviance)
cat("McFadden pseudo-R²:", round(mcfadden,3), "\n")
cat("(0.2-0.4 = good fit for logistic models)\n")

# —— Odds Ratios (exponentiated coefficients) ——————————————————————————————————
cat("\n—— Odds Ratios with 95% Confidence Intervals ——\n")
results logit <- tidy(logit model, conf.int=TRUE, exponentiate=TRUE) %>%
  mutate(across(where(is.numeric), ~round(.,4)))
print(results_logit)

# —— Classification performance ————————————————————————————————————————————————
cat("\n—— Classification performance (threshold = 0.5) ——\n")
predicted class <- as.integer(fitted(logit model) >= 0.5)
actual_class    <- logistic_df[[binary_outcome]]
conf matrix     <- table(Predicted=predicted class, Actual=actual class)
print(conf matrix)

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
cat("Accuracy:", round(accuracy*100,1), "% \n")

# —— Save results ——————————————————————————————————————————————————————————————
write csv(results logit,
  paste0("output/tables/04_logit_OR_", ANALYSIS_LABEL, ".csv"))
cat("Script #7 complete.\n")
```

Assumption Check Summary Table

Use this table to document the result of every assumption check after running Script #7. Record your findings before reporting results.

Assumption	Test / Method	Pass criteria	Your result	Action if violated
LINEAR REGRESSION				
Linearity	Residuals vs Fitted plot	Random scatter, no curve	[ ] Pass [ ] Fail	Log-transform the outcome or use polynomial terms
Homoscedasticity	Breusch-Pagan test, Scale-Location plot	$p > 0.05$, flat red line	[ ] Pass [ ] Fail	Use heteroscedasticity-robust standard errors (sandwich package)

Normality of residuals	Shapiro-Wilk test, Q-Q plot	$p > 0.05$, points on diagonal	[ ] Pass [ ] Fail	Log-transform outcome
No influential outliers	Cook's Distance	No observations above 4/n threshold	[ ] Pass [ ] Fail	Run sensitivity analysis with and without flagged observations
No multicollinearity	VIF (all predictors)	All VIF < 5	[ ] Pass [ ] Fail	Remove highest VIF predictor and re-run
LOGISTIC REGRESSION				
Sample size (EPV)	Events per variable calculation	EPV ≥ 10	[ ] Pass [ ] Fail	Reduce number of predictors
No complete separation	Large coefficient check	No log-odds > 10	[ ] Pass [ ] Fail	Remove or combine the separating predictor
Goodness of fit	Hosmer-Lemeshow test	$p > 0.05$	[ ] Pass [ ] Fail	Revise model by adding or removing predictors
No multicollinearity	VIF (same as linear)	All VIF < 5	[ ] Pass [ ] Fail	Remove highest VIF predictor and re-run
BOTH REGRESSIONS				
Independence of observations	Study design review; check for spatial clustering if relevant	Each geographic area represents one independent observation; no obvious duplicates or nested observations	[ ] Pass [ ] Fail	Spatial autocorrelation: consider Moran's I if needed

5. Export and Data Dictionary

At the end of your analysis workflow, make sure these files exist in your output/ folder before moving to Part 3.

Required Output Files

Folder	File	Description
DATA FOLDER		
data/clean/	01_acs_clean.csv	ACS 5-year cleaned county data
	02_places_clean.csv	CDC PLACES health outcomes
	03a_svi_clean.csv	SVI 2022 county data
	03b_eji_county_clean.csv	EJI 2024 aggregated to county
	04_hpsa_clean.csv	HRSA HPSA shortage designations
	05a_usda_food_access_clean.csv	USDA food access atlas
	05b_chr_clean.csv	County Health Rankings 2025
	05c_nces_ccd_clean.csv	NCES CCD enrollment
data/master/	sdoh_[state]_master.csv	Final merged master dataset (Script #6)

OUTPUT FOLDER		
output/figures/	02_correlation_[ANALYSIS_LABEL].png	Correlation matrix plot
	03_lm_diagnostics_[ANALYSIS_LABEL].png	Linear regression diagnostic plots
output/tableau/	sdoh_[state]_tableau.csv	Master dataset for Tableau
output/tables/	01_descriptive_[ANALYSIS_LABEL].csv	Descriptive statistics
	03_lm_results_[ANALYSIS_LABEL].csv	Linear regression coefficients and CIs
	03_lm_results_[ANALYSIS_LABEL].csv	Robust standard errors results, if required by the Breusch-Pagan test
	04_logit_OR_[ANALYSIS_LABEL].csv	Logistic regression odds ratios
DICTIONARY FOLDER		
dictionary/	data_dictionary.csv	Variable labels, sources, and years
ROOT FOLDER		
sdoh_toolkit/	README.txt	Project documentation

Data Dictionary Template
Every project must include a data dictionary listing every variable in the master CSV. This is required for your GitHub repository and for open scholarship compliance.

Generating Your Project README
The README code automatically generates a plain-text documentation file that records everything needed to reproduce your analysis (e.g., who ran it, when, which data were used, which variables were selected, which software versions were installed, and where the output files are saved). This is a core open scholarship requirement for any publicly shared research project.

```
# ── Generate README.txt automatically ──────────────────────────────────────────
sink("README.txt")
cat("=== SDOH TOOLKIT — PROJECT README ===\n\n")
cat("Author:           [Your name]\n")
cat("Institution:      [Your institution]\n")
cat("Date completed:   ", format(Sys.Date(), "%Y-%m-%d"), "\n")
cat("Research question:[One sentence]\n\n")

cat("--- Analysis configuration ---\n")
cat("State:            ", STATE_LABEL, "\n")
cat("Geography:        County level\n")
cat("Outcome variable: ", OUTCOME_VAR, "\n")
cat("Predictors:       ", paste(PREDICTOR_VARS, collapse=", "), "\n")
cat("Analysis label:   ", ANALYSIS_LABEL, "\n\n")

cat("--- Script run order ---\n")
cat("01_acs.R → 02_places.R → 03_svi_eji.R → 04_hrsa.R\n")
cat("→ 05_usda_nces_chr.R → 06_merge_all.R → 07_analysis.R\n\n")

cat("--- Data download dates ---\n")
cat("ACS 2020-2024:    [date]\n")
cat("CDC PLACES 2025:  [date]\n")
cat("SVI 2022:         [date]\n")
cat("EJI 2024:         [date]\n")
cat("HRSA AHRF 2024-25:[date]\n")
cat("USDA Atlas 2019:  [date]\n")
cat("CHR&R 2025:       [date]\n")
cat("NCES CCD 2024-25: [date]\n\n")
```

```
cat("--- Output files ---\n")
cat("Master CSV:    data/master/sdoh_", STATE_LABEL, "_master.csv\n")
cat("Tableau CSV:   output/tableau/sdoh_", STATE_LABEL, "_tableau.csv\n")
cat("Descriptive:   output/tables/01_descriptive_", ANALYSIS_LABEL, ".csv\n")
cat("LM results:    output/tables/03_lm_results_", ANALYSIS_LABEL, ".csv\n")
cat("Logit results: output/tables/04_logit_OR_", ANALYSIS_LABEL, ".csv\n")
cat("Correlation:   output/figures/02_correlation_", ANALYSIS_LABEL, ".png\n")
cat("Diagnostics:   output/figures/03_lm_diagnostics_", ANALYSIS_LABEL, ".png\n\n")

cat("--- GitHub repository ---\n")
cat("Repository URL: [your GitHub URL]\n")
cat("Visibility:     Public\n\n")

cat("--- R environment ---\n")
cat("R version:", R.version$major, ".", R.version$minor, "\n")
cat("OS:", Sys.info()["sysname"], Sys.info()["release"], "\n\n")

cat("--- Package versions ---\n")
pkgs <- c("tidycensus","tidyverse","haven","readxl",
          "janitor","skimr","corrplot","car","broom","lmtest",
          "ResourceSelection","sandwich","devtools")
for(p in pkgs){
  if(requireNamespace(p, quietly=TRUE)){
    cat(p, ":", as.character(packageVersion(p)), "\n")
  }
}
sink()
cat("README.txt saved.\n")
```

Create `dictionary/data_dictionary.csv`

Every project must include a data dictionary listing every variable in the master CSV. This is required for your GitHub repository and for open scholarship compliance. Run this code after Script #6 has completed successfully.

```
# ── Create data dictionary from master file ──────────────────────────────────

library(tidyverse)

master <- read_csv("data/master/sdoh_il_master.csv")

# ── Base dictionary from column names ────────────────────────────────────────
data_dict <- tibble(
variable_name = names(master),
label = c(
"5-digit county FIPS code (character)",          # fips
"County name (ACS format, includes state)",       # county_name
"Poverty rate (% persons below poverty line)",    # poverty_rate
"Median household income ($)",                    # median_hh_income
"Unemployment rate (% civilian labor force)",     # unemployment_rate
"SNAP participation rate (% households)",         # snap_rate
"Rent cost burden rate (% renters paying 30%+ income on rent)", # rent_burden_rate
"% adults 25+ with less than HS diploma",        # pct_no_hs
"% adults 25+ with HS diploma or equivalent",    # pct_hs_grad
"% adults 25+ with bachelor's degree or higher", # pct_bachelor_plus
"Uninsured rate (% population, ACS-derived)",    # uninsured_rate
"Adult obesity prevalence (%, CDC PLACES 2025)", # obesity_pct
```

```
"Depression prevalence (%, CDC PLACES 2025)",    # depression pct
"Diagnosed diabetes prevalence (%, CDC PLACES 2025)", # diabetes pct
"COPD prevalence (%, CDC PLACES 2025)",          # copd_pct
"Current smoking prevalence (%, CDC PLACES 2025)", # smoking pct
"Mammography use rate (%, CDC PLACES 2025)",     # mammography pct
"Physical inactivity prevalence (%, CDC PLACES 2025)", # inactivity_pct
"Current asthma prevalence (%, CDC PLACES 2025)", # asthma pct
"Uninsured % (CDC PLACES 2025, Access2 measure)", # uninsured places
"High blood pressure prevalence (%, CDC PLACES 2025)", # hypertension_pct
"Colorectal cancer screening rate (%, CDC PLACES 2025)", # colorectal screen pct
"SVI overall percentile (0-1; higher=more vulnerable, CDC/ATSDR 2022)", #
svi_overall
"SVI Theme 1: socioeconomic status percentile",  # svi socioeco
"SVI Theme 2: household characteristics percentile", # svi hh char
"SVI Theme 3: racial/ethnic minority and language percentile", # svi minority
"SVI Theme 4: housing type and transportation percentile", # svi_housing
"% below 150% poverty (SVI component)",          # pct poverty
"% unemployed (SVI component)",                  # pct_unemp_svi
"% uninsured (SVI component)",                   # pct_uninsured_svi
"EJI overall county mean percentile (CDC/ATSDR 2024)", # eji overall
"EJI Environmental Burden mean percentile",      # eji_env_burden
"EJI Social Vulnerability mean percentile",      # eji_social_vuln
"EJI Health Vulnerability mean percentile",      # eji health vuln
"Number of census tracts aggregated for EJI",    # n_tracts
"Primary care shortage designation (0=none, 1=partial, 2=full)", # hpsa_prim_care
"Dental shortage designation (0=none, 1=partial, 2=full)", # hpsa dental
"Mental health shortage designation (0=none, 1=partial, 2=full)", #
hpsa_mental_health
"Any shortage designation (1=yes, 0=no)",        # hpsa any shortage
"Primary care shortage binary (1=yes, 0=no)",    # hpsa_prim_care_designated
"% census tracts that are low-income AND low-access (USDA)", # pct lila tracts
"Mean food access share (USDA)",                 # mean dist supermarket
"% housing units lacking vehicle and far from supermarket", # pct_no_vehicle_far
"Premature death rate (CHR&R 2025)",             # premature_death
"Avg physically unhealthy days per month (CHR&R 2025)", # poor health days
"Avg mentally unhealthy days per month (CHR&R 2025)", # poor_mental_days
"Social associations per 100,000 population (CHR&R 2025)", # social_associations
"Voter turnout % (CHR&R 2025, presidential election)", # voter turnout
"Adult smoking % (CHR&R 2025)",                  # adult_smoking
"Adult obesity % (CHR&R 2025)",                  # adult_obesity
"Food insecurity % (CHR&R 2025)",                # food insecurity
"Number of school districts in county (NCES CCD)", # n_districts
"Total student enrollment in county",            # total_enrollment
"% students eligible for free lunch",            # pct free lunch
"% students eligible for reduced-price lunch"    # pct_reduced_lunch
),
  source = c(
    "U.S. Census Bureau", "U.S. Census Bureau",
    rep("ACS 5-Year 2020-2024", 9),
    rep("CDC PLACES 2025", 11),
    rep("CDC/ATSDR SVI 2022", 8),
    rep("CDC/ATSDR EJI 2024", 5),
    rep("HRSA AHRF 2025", 5),
    rep("USDA Food Access Atlas 2019", 3),
    rep("CHR&R 2025", 8),
    rep("NCES CCD 2024", 4)
  ),
```

```
  year = c(
    "2024", "2024",
    rep("2020-2024", 9),
    rep("2021-2022 BRFSS", 11),
    rep("2022", 8),
    rep("2024", 5),
    rep("2025", 5),      #HPSA
    rep("2019", 3),      # USDA
    rep("2025", 8),      # CHR&R
    rep("2024", 4)       # NCES
  )
)

write_csv(data_dict, "dictionary/data_dictionary.csv")
cat("Data dictionary saved with", nrow(data_dict), "variables.\n")
```

6. Part 2 Checklist

Complete all items before moving to Part 3. Items 1-5 are required. Items 6 and above are best practices for open scholarship.

- ☐ I have installed R 4.3+, RStudio, and all required packages (Section 1B).
- ☐ I have obtained a Census API key and stored it with `census_api_key()` (Section 1C).
- ☐ I have created the project folder structure and all required subdirectories (Section 1D).
- ☐ Script #1 runs without errors and produces 01_acs_clean.csv with the expected number of rows for your chosen geography (e.g., 102 for IL counties, approximately 3,000 for IL census tracts).
- ☐ Scripts #2-#5 run without errors and produce clean CSV files in data/clean/.
- ☐ Script #6 produces `sdoh_[state]_master.csv` with the expected number of rows for your chosen geography and no rows dropped during merges.
- ☐ Script #7 produces descriptive statistics, correlation matrix, unadjusted and adjusted regression results, assumption check outputs, and follow-up analyses where required.
- ☐ I have reviewed VIF values and confirmed no multicollinearity problem (VIF < 5 for all predictors).
- ☐ I have completed the Assumption Check Summary Table and documented Pass/Fail for all linear and logistic regression assumptions.
- ☐ If Breusch-Pagan $p < 0.05$, I have applied robust standard errors using the sandwich package.
- ☐ If Cook's Distance flagged influential observations, I have run and reviewed the sensitivity analysis.
- ☐ `output/tableau/sdoh_[state]_tableau.csv` exists and is ready for Tableau (Part 3).
- ☐ `dictionary/data_dictionary.csv` exists and lists all variables with source and year.
- ☐ README.txt has been generated and saved to the project root folder. GitHub repository setup and open sharing will be completed in Part 3. If this toolkit is being used as part of a course, submit the repository URL according to your instructor’s directions.

If any script produces an error, check: (1) file paths match your folder structure exactly, (2) FIPS columns are 5-digit character strings, (3) package versions are current — run `update.packages()`, and (4) your Census API key is loaded. Run `readRenviron("~/.Renviron")` if `sys.getenv("CENSUS_API_KEY")` returns blank. Do not move to Part 3 until the master CSV has the expected number of rows for your chosen geography with no critical missing data.

Ready for the next step?

Part 3 covers building and publishing your Tableau Public dashboard as an open, freely accessible resource.

Part 3: Tableau, Repository & Open Sharing →